

Текст

Текст

Содержание

1. Некоторые модули в Python.....	5
1.1. Математический модуль math.....	5
1.2. Модуль random.....	9
1.3. Построение графиков с matplotlib	11
1.4. Библиотека NumPy.....	26
1.5. Пакет SciPy	29
2. Задачи на matplotlib.....	30
2.1. Примеры.....	30
2.2. Условия задач.....	34
3. Декораторы	42
3.1. Написание пользовательского декоратора.....	42
3.2. Применение декоратора к функции.....	43
3.3. Переменное число аргументов функции	45
3.4. Декораторы с параметрами	46
3.5. Декораторы классов	47
4. ООП в Python.....	47
4.1. Объектно-ориентированное программирование.....	47
4.2. Написание пользовательского класса.....	49
4.3. Специальные (магические) методы.....	50
4.4. Инкапсуляция, наследование и полиморфизм в Python	54
4.5. Свойства (геттеры и сеттеры)	57
4.6. Абстрактные классы	58
4.7. Статические поля и методы	59

5. Аннотации.....	61
6. Исключения	62
7. Комплексные числа	64
8. Длинная арифметика.....	65
9. Множественный выбор match-case.....	65
10. Перечисления Enum	68
11. Задачи на ООП и декораторы	70
11.1. Пример.....	70
11.2 Условия задач	73
Список литературы	78
Приложение 1. Код пользовательского контейнера	80
Приложение 2. Демонстрация property	82
Приложение 3. Оператор match-case с классами	82

1. Некоторые модули в Python

1.1. Математический модуль math

Модуль предоставляет обширный функционал для работы с числами. Он содержит: тригонометрические и обратные тригонометрические функции; факториал; функции перевода градусов в радианы и обратно; логарифм; квадратный корень; функцию вычисления абсолютного значения (модуля) числа; округление и другие. Пример использования данного модуля представлен на рисунке 1.1.

```
1 import math
2
3 print(math.factorial(6)) # 720
4 print(math.log(81, 3))  # 4.0
```

Рис. 1.1. Импорт math, печать факториала и логарифма

К функциям **округления** относятся (рис. 1.2):

- ceil округляет число до ближайшего целого в большую сторону;
- floor округляет число до ближайшего целого в меньшую сторону;
- trunc усекает значение до целого.

```
1 print(math.ceil(6.3)) # 7
2 print(math.ceil(-6.3)) # -6
3 print(math.floor(6.3)) # 6
4 print(math.floor(-6.3)) # -7
5 print(math.trunc(6.3)) # 6
6 print(math.trunc(-6.3)) # -6
```

Рис. 1.2. Пример вызова функций округления

На этом и дальнейших рисунках раздела принимается что модуль math уже подключен выше.

Для привычного округления (на самом деле для «банковского») используется метод round не относящийся к модулю math.

Функция **fabs** (рис. 1.3) используется для вычисления абсолютного значения числа. Она всегда возвращает число с плавающей точкой.

```
1 print(math.fabs(5)) # 5.0
2 print(math.fabs(-5)) # 5.0
3 print(math.fabs(-5.5)) # 5.5
```

Рис. 1.3. Вычисление модуля

Функция **factorial** принимает положительное целое число и выводит его факториал.

Функция **fmod** возвращает остаток от деления. Для чисел типа `float` данная функция является предпочтительнее чем оператор `%`, который в свою очередь является предпочтительным для чисел типа `int`.

Функция **frexp** возвращает мантиссу и показатель степени в виде пары (m, e) любого числа x , решая следующее уравнение: $x = m * 2^e$. Мантисса m представляет собой число с плавающей запятой, а показатель e представляет собой целочисленное значение. Значение m вычисляется таким способом, чтобы его модуль лежал в полуинтервале $[0.5; 1)$. Данная функция используется тогда, когда представление чисел типа `float` не должно зависеть от архитектуры машины.

Функция **ldexp** возвращает значение равное $m * 2^e$, то есть является обратной к функции `frexp`.

Функция **modf** (рис. 1.4) разделяет число на дробную часть f и целую часть w и возвращает пару вещественных чисел в виде кортежа (f, w) .

```
1 print(math.modf(123.45)) # (0.450000000000000284, 123.0)
```

Рис. 1.4. Вызов функции `modf`

Функция **copysign** возвращает число с абсолютным значением первого аргумента, но со знаком второго. Возвращаемый результат всегда типа `float`.

Функция **isclose** (рис. 1.5) позволяет проверить на равенство вещественные числа с учетом погрешности. Точность сравнения указывается в параметрах `rel_tol` (задает максимально допустимую погрешность в виде доли) и `abs_tol` (задает абсолютную максимально допустимую погрешность).

Функция **lcm** возвращает наименьшее общее кратное указанных целочисленных аргументов. Если какой-либо из аргументов равен нулю, то возвращается нуль.

```

1  x = 7
2  y = 7.069
3
4  # Обычное сравнение
5  print(x == y) # False
6
7  # Модуль разности между x и y должен быть не больше 0.01
8  print(math.isclose(x, y, abs_tol=0.01)) # False
9
10 # Разница между x и y должна быть не больше 1%
11 print(math.isclose(x, y, rel_tol=0.01)) # True

```

Рис. 1.5. Использование функции isclose

Функция **gcd** возвращает наибольший общий делитель указанных целочисленных аргументов.

Функция **nextafter** (рис. 1.6) возвращает следующее значение float после аргумента 1 по направлению к аргументу 2.

```

1  print(math.nextafter(6.2, 100)) # 6.2000000000000001
2  print(math.nextafter(6.2, 0))   # 6.199999999999999

```

Рис. 1.6. Вызов функции nextafter

Функция **fsum** (рис. 1.7) вычисляет точную сумму значений с плавающей точкой в итерируемом объекте (например, последовательности).

```

1  x = (6.2, 20, 4.1522)
2  print(math.fsum(x)) # 30.3522

```

Рис. 1.7. Вычисление суммы кортежа с помощью модуля math

Функция **exp** принимает один вещественный параметр возвращает экспоненту в данной степени.

Функция **log** находит логарифм числа по заданному основанию. Если оно не указано, то вычисляется натуральный логарифм.

Функция **pow** возводит число в степень.

Функция **sqrt** вычисляет квадратный корень.

В math есть следующие **тригонометрические функции** (вычисления выполняются в радианах):

- sin возвращает синус;
- cos возвращает косинус;
- tan возвращает тангенс;

- `asin` возвращает арксинус (обратный синус);
- `acos` возвращает арккосинус (обратный косинус);
- `atan` возвращает арктангенс (обратный тангенс);
- `sinh` возвращает гиперболический синус;
- `cosh` возвращает гиперболический косинус;
- `tanh` возвращает гиперболический тангенс;
- `asinh` возвращает обратный гиперболический синус;
- `acosh` возвращает обратный гиперболический косинус;
- `atanh` возвращает обратный гиперболический тангенс.

Кроме того, модуль содержит **функции преобразования углов**:

- `degrees` конвертирует радианы в градусы;
- `radians` конвертирует градусы в радианы.

Также модуль содержит четыре константы. Константа **`pi`** содержит число $\pi = 3.141592653589793$. Константа **`e`** содержит экспоненту (число Эйлера), равную 2.718281828459045.

Константа **`inf`** возвращает положительную бесконечность, значение которой относится к типу `float` и может присутствовать в математических выражениях (рис. 1.8).

```
1 print(math.inf/2) # inf
```

Рис. 1.8. Деление бесконечности пополам

Константа **`nan`** возвращает значение «не число» (not a number) которое относится к типу `float` и также может присутствовать в математических выражениях. Оно не считается равным никакому другому числовому значению, включая само себя (рис. 1.9).

```
1 print(math.inf * 0) # nan
2 print(math.nan == math.nan) # False
```

Рис. 1.9. Константа `nan` в модуле `math`

Обычно получение в качестве результата `nan` означает, что где-то допущена ошибка.

1.2. Модуль random

Модуль random предоставляет функции для:

- генерации случайных чисел (randint, random, uniform);
- случайного выбора элементов последовательности (choice);
- перемешивания последовательности (shuffle) и другие...

Пример подключения и использования представлен на рисунке 1.10.

```
1 from random import randint, uniform
2
3 i = randint(1, 50) # Целое
4 f = uniform(1, 50) # Вещественное
```

Рис. 1.10. Использование модуля random

Метод **randint** в Python возвращает случайное целочисленное значение между двумя нижними и верхними пределами (включая оба ограничения), указываемыми как два целочисленных параметра. То есть в случае из примера на рисунке 1.10, значение *i* будет принадлежать отрезку [1; 50].

Метод **random** возвращает случайное число с плавающей точкой в промежутке [0.0, 1.0].

Метод **randrange** (рис. 1.11) используется для генерации случайного числа в диапазоне, указанном в его параметрах: первый — левая граница, второй — правая граница (не включена), третий — ширина (шаг), которая используется для пропуска числа в диапазоне. Последний параметр является необязательным, и, если он не указан, принимает значение по умолчанию равное единице.

```
1 print(random.randrange(0, 5, 2)) # 0, 2 или 4
2 print(random.randrange(1, 20, 5)) # 1, 6, 11 или 16
```

Рис. 1.11. Пример работы с randrange

На этом и дальнейших рисунках раздела принимается что модуль random уже подключен выше.

Метод **uniform** — возвращает случайное число с плавающей точкой в указанном диапазоне, причем нижняя граница включена в этот диапазон, а верхняя нет.

В модуле `random` есть функция **choice** (рис. 1.12), используемая для выбора случайного элемента из списка или другой последовательности.

```
1 s = 'Если тебя обидели незаслуженно вернись и заслужи'
2 print(random.choice(s)) # Случайный символ
```

Рис. 1.12. Выбор случайного символа из строки

Функция **sample** (рис. 1.13) используется, когда требуется выбрать несколько элементов из заданной последовательности. Метод возвращает список уникальных элементов, которые были выбраны из указанной в параметрах последовательности. Выбираемое количество элементов также задается параметром.

```
1 s = 'Мы построим свои триноты. Но только с 4 ногами'
2 print(random.sample(s, 5))
3 # Возможный результат:
4 # ['г', 'к', 'н', 'о', '.']
```

Рис. 1.13. Выбор нескольких случайных символов из строки

Функция **choices** также используется, когда требуется выбрать несколько случайных элементов из заданной последовательности, но позволяет повторять несколько раз один и тот же элемент. То есть, по сути, она заменяет вызов функции `choice` в цикле.

Функция **seed** используется для инициализации генератора псевдослучайных чисел в Python. Модуль `random` использует значение из `seed`, или отправной точки как основу для генерации случайного числа. Если значения `seed` нет в наличии, тогда система будет отталкиваться от текущего времени.

Функция **shuffle** используется для перемешивания данных списка или другой последовательности. Она смешивает элементы списка на месте, то есть изменяет сам список (рис. 1.14).

```
1 # Генерация списка из натуральных чисел
2 arr = list(range(1,16))
3 # Перемешивание
4 random.shuffle(arr)
5 print(arr)
6 # Возможный результат
7 # [1, 2, 6, 14, 5, 4, 9, 10, 3, 11, 7, 13, 15, 12, 8]
```

Рис. 1.14. Генерация и перемешивание списка

Важно то, что функция `shuffle` применяется для изменяемых последовательностей, то есть попытка применить его к строке или кортежу вызовет ошибку.

Функция **`triangular`** используется в генерации случайных вещественных чисел с плавающей точкой. При генерации значения применяется треугольное распределение вероятности.

1.3. Построение графиков с `matplotlib`

`Matplotlib` — это набор дополнительных модулей на языке Python для визуализации данных. В нем можно построить двумерные и трехмерные графики.

Установка `matplotlib` для ОС Windows возможна через командную строку:

```
python -m pip install -U pip
pip install matplotlib
```

Для построения несложных графиков достаточно подключить модуль `pyplot` как на рисунке 1.15.

```
1 import matplotlib.pyplot as p
```

Рис. 1.15. Подключение `pyplot`

Функция **`plot`** служит для построения графика и, в качестве параметров, принимает «иксы» и «игреки».

Функция **`show`** выводит построенное изображение (рис. 1.16).

```
1 p.plot(list(range(5)), [1, 3, 3, 1, -5])
2 p.show()
```

Рис. 1.16. Код для отрисовки простого графика

В результате будет получен график, представленный на рисунке 1.17.

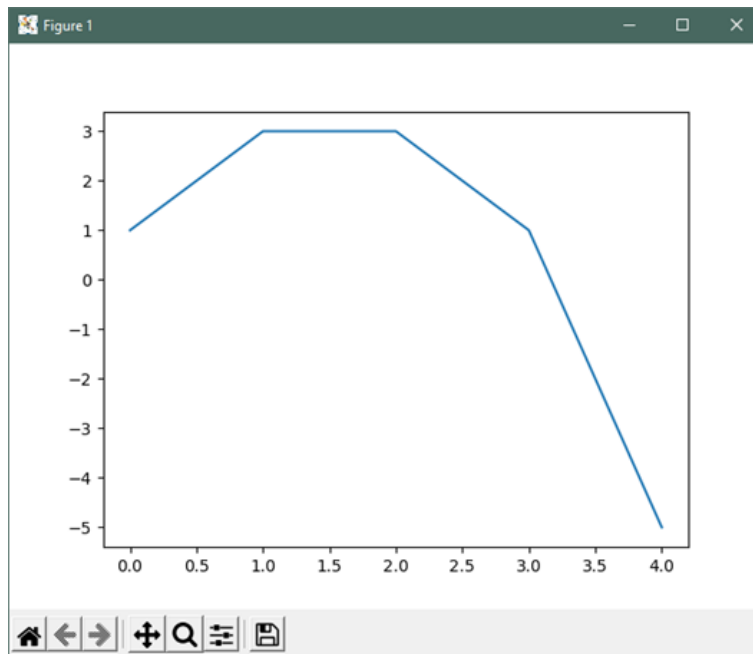


Рис. 1.17. График ломанной, заданной пятью точками

Добавляя в `plot` через запятую пары значений можно построить несколько графиков одновременно. Либо можно вызвать `plot` нужное число раз. Код примера представлен на рисунке 1.18, а результат — на 1.19.

```

1  x = tuple(range(5))
2  y1 = (1, 3, 3, 1, -5)
3  y2 = (5, 2, 4, 0, 1)
4  p.plot(x, y1, x, y2)
5  p.show()

```

Рис. 1.18. Код для вывода двух ломанных

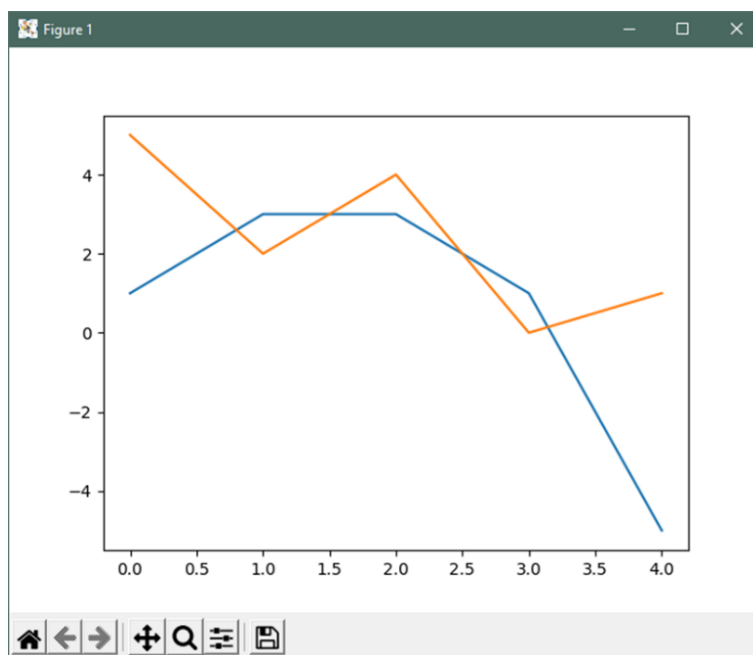


Рис. 1.19. Вывод двух ломанных

Цвет линий задается параметром `color` функции `plot`, значениями которого могут быть как коды цветов, так и их названия (рис. 1.20).

```
1 p.plot(x, y1, color = (1.0, 0.0, 1.0)) # в долях RGB
2 p.plot(x, y2, color = '#df8856')      # html-код цвета
```

Рис. 1.20. Код для задания цвета линий

Базовые цвета могут быть указаны одной буквой:

- b — синий;
- g — зеленый;
- r — красный;
- c — циан;
- m — манжета;
- y — желтый;
- k — черный;
- w — белый.

Цвета, задаваемые полными названиями представлены на рисунке 1.21.

black	k	dimgray	dimgray
gray	grey	darkgray	darkgrey
silver	lightgray	lightgray	gainsboro
whitesmoke	w	white	snow
rosybrown	lightcoral	indianred	brown
firebrick	maroon	darkred	r
red	mistyrose	salmon	tomato
darksalmon	coral	orangered	lightsalmon
sienna	seashell	chocolate	saddlebrown
sandybrown	peachpuff	peru	linen
bisque	darkorange	burlywood	antiquewhite
tan	navajowhite	blanchedalmond	papayawhip
moccasin	orange	wheat	oldlace
floralwhite	darkgoldenrod	goldenrod	cornsilk
gold	lemonchiffon	khaki	palegoldenrod
darkkhaki	ivory	beige	lightyellow
lightgoldenrodyellow	olive	y	yellow
olivedrab	yellowgreen	darkolivegreen	greenyellow
chartreuse	lawngreen	honeydew	darkseagreen
palegreen	lightgreen	forestgreen	limegreen
darkgreen	g	green	lime
seagreen	mediumseagreen	springgreen	mintcream
mediumspringgreen	mediumaquamarine	aquamarine	turquoise
lightseagreen	mediumturquoise	azure	lightcyan
paleturquoise	darkslategray	darkslategrey	teal
darkcyan	c	aqua	cyan
darkturquoise	cadetblue	powderblue	lightblue
deeppskyblue	skyblue	lightskyblue	steelblue
aliceblue	dodgerblue	lightslategray	lightslategray
slategray	slategrey	lightsteelblue	cornflowerblue
royalblue	ghostwhite	lavender	midnightblue
navy	darkblue	mediumblue	b
blue	slateblue	darkslateblue	mediumslateblue
mediumpurple	rebeccapurple	blueviolet	indigo
darkorchid	darkviolet	mediumorchid	thistle
plum	violet	purple	darkmagenta
m	fuchsia	magenta	orchid
mediumvioletred	deeppink	hotpink	lavenderblush
palevioletred	crimson	pink	lightpink

Рис. 1.21. Названия цветов в Python

Параметр `linewidth` определяет **толщину линий** (рис. 1.22).

```

1 p.plot(x, y1, linewidth = 6)
2 p.plot(x, y2, linewidth = 3)

```

Рис. 1.22. Код для настройки толщины линии

Тип линии задается параметром `linestyle` (рис. 1.23). Линия может быть:

- сплошной ('-', 'solid');
- штриховой ('--', 'dashed');
- штрих-пунктирной или «тире-точка» ('-.', 'dashdot');
- пунктирной или «из точек» (':', 'dotted');
- не отображаться ('None', '', '').

```

1 p.plot(x, y1, linestyle = '--')
2 p.plot(x, y2, linestyle = 'dashdot')

```

Рис. 1.23. Код задания разных типов линий

На рисунке 1.24 показан график с заданными цветом, толщиной и типом линии.

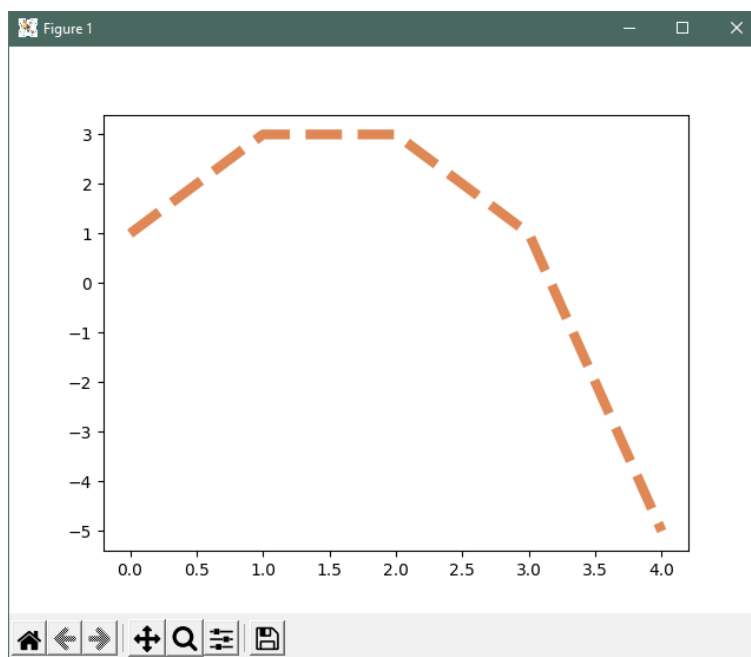


Рис. 1.24. Рисунок с настроенной линией

Можно указать **надпись** над графиком (заголовок) с помощью функции `title`. Чтобы **подписать** одну либо обе **оси** существуют функции `xlabel` и `ylabel` (рис. 1.25 и 1.26).

```

1 p.title("Два графика")
2 p.xlabel("Время")
3 p.ylabel("Скорость")

```

Рис. 1.25. Код для задания подписей

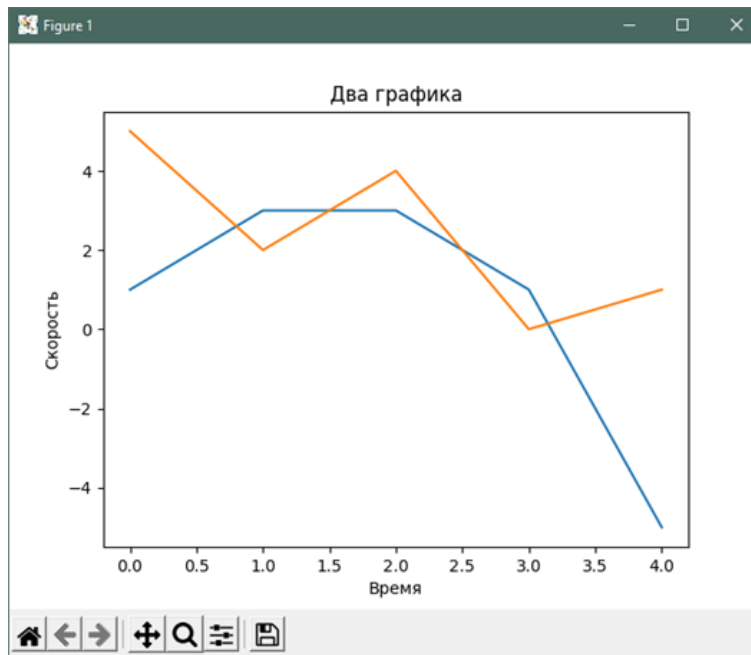


Рис. 1.26. Рисунок с подписями

Чтобы просто включить или выключить **сетку** достаточно вызвать функцию `grid`, указав булевский аргумент — отображать сетку или нет. Цвет, толщина и тип линий сетки можно настраивать, аналогично тому, как это настраивается для самих графиков.

Чтобы настроить пользовательские значения вдоль осей (метки), а вместе с ними и ячейки сетки, используются функции `xticks` и `yticks` (рис. 1.27 и 1.28).

```

1 p.grid(True) # Сетка
2 p.xticks([0, 4/3, 8/3, 4])
3 p.yticks(range(-4, 5, 1))

```

Рис. 1.27. Код для настройки сетки и меток по осям

Нарисовать оси можно с помощью методов `axhline` (для OX) и `axvline` (для OY).

Figure — это контейнер самого верхнего уровня. Таких контейнеров может быть несколько, каждый из которых может содержать несколько контейнеров `Axes`.

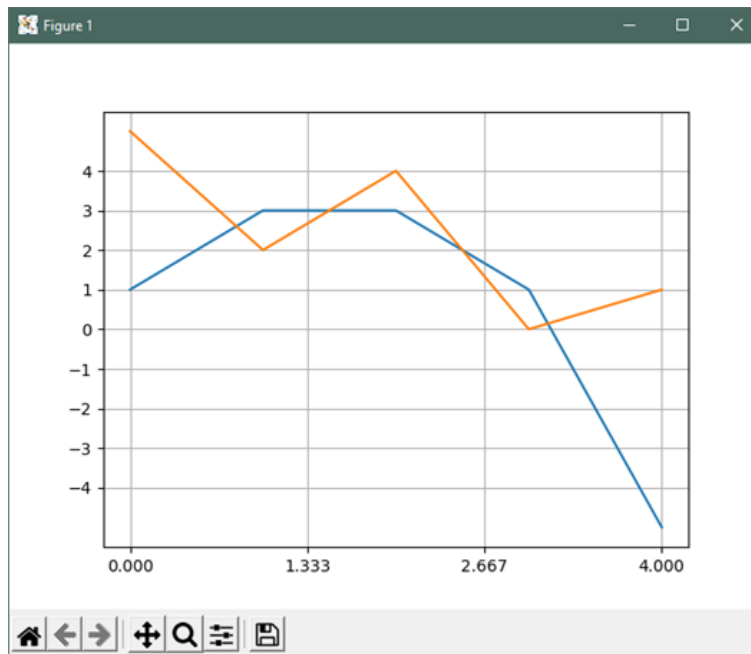


Рис. 1.28. График с пользовательскими значениями и сеткой

Axes — это область, на которой отображаются графики, а также все вспомогательные атрибуты: линии сетки, метки, указатели и т.д. (рис. 1.29 и рис. 1.30).

```

1  fig = p.figure()
2  fig.patch.set_facecolor('#f66') # красный
3
4  ax = p.axes()
5  ax.set_facecolor('#6f6')       # зеленый
6
7  p.plot(x, y1, color='#66f')    # синий

```

Рис. 1.29. Код для раскраски figure, axes и линии

Чтобы разместить несколько областей для рисования графиков (axes) рядом, надо воспользоваться одним из двух способов:

1. Метод **add_subplot**, который принимает в качестве параметров количество строк и столбцов, на которые мысленно нужно разделить окно, и порядковый номер в таком делении (рис. 1.31).

2. Функция **subplot2grid**, которая принимает в качестве параметров количество строк и столбцов, на которые мысленно нужно разделить окно, координаты нужного места при таком разбиении, а также количество занимаемых «ячеек» такой мысленной сетки (рис. 1.32).

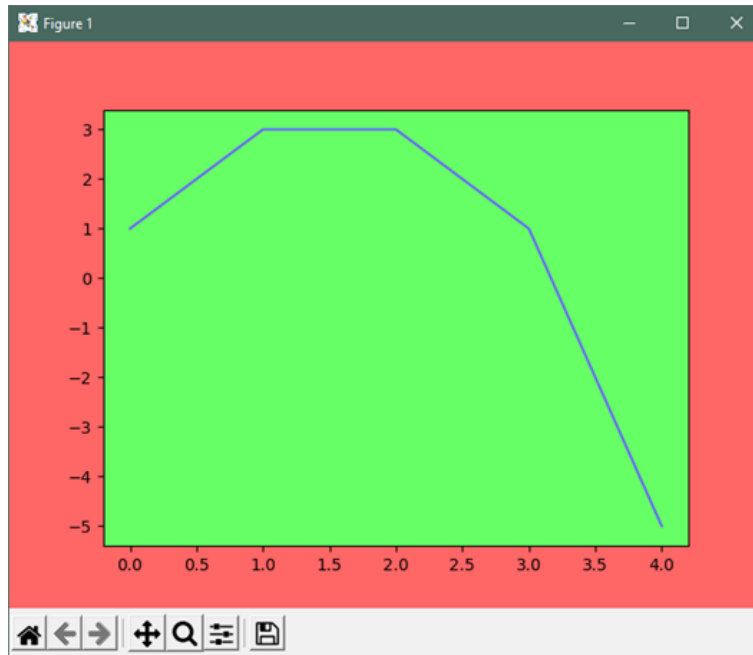


Рис. 1.30. Окно, где figure, axes и линия раскрашены разными цветами

```

1  ax0 = fig.add_subplot(1, 2, 1)
2  ax1 = fig.add_subplot(2, 2, 2)
3  ax2 = fig.add_subplot(2, 2, 4)

```

Рис. 1.31. Пример с add_subplot

```

1  ax0 = p.subplot2grid((2, 2), (0, 0), 2, 1)
2  ax1 = p.subplot2grid((2, 2), (0, 1), 1, 1)
3  ax2 = p.subplot2grid((2, 2), (1, 1), 1, 1)

```

Рис. 1.32. Пример с subplot2grid

Результат выполнения кода для обоих примеров представлен на рисунке 1.33.

Для каждой из областей рисования можно задать свои **границы обзора** методами `set_xlim` и `set_ylim` (рис. 1.34 и рис. 1.35).

На рисунке 1.35 стрелочками показано что определяют эти методы. Для наглядности можно сравнить с той же ломаной без указания границ (то есть со значениями по умолчанию) на рисунке 1.17, на котором график «вписан» в оси, то есть буквально занимает всю ширину и всю высоту осей, в которых он рисуется.

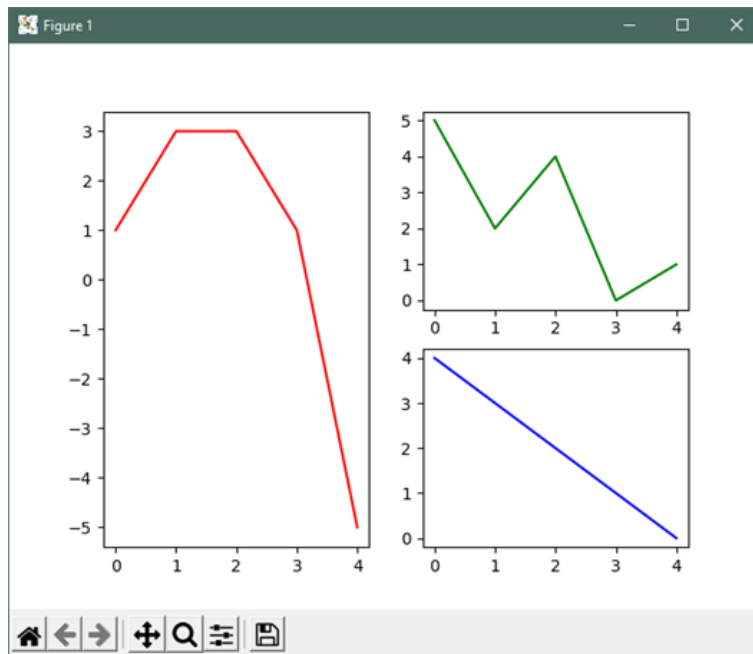


Рис. 1.33. Три графика в разных осях на одном окне

```

1  fig = p.figure()
2  ax = fig.add_subplot(1, 1, 1)
3  ax.set_xlim(-5, 5)
4  ax.set_ylim(-5, 5)

```

Рис. 1.34. Код для выбора рисуемой части плоскости

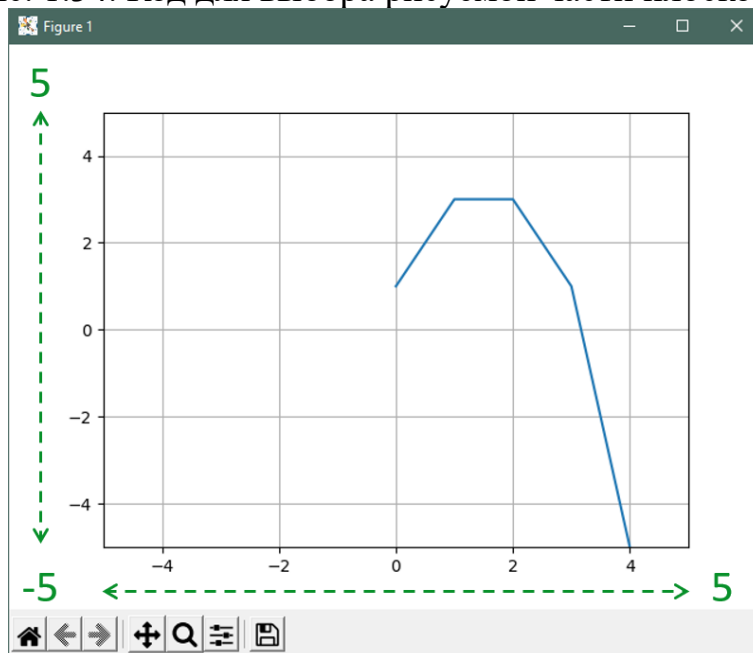


Рис. 1.35. Ломаная в указанных диапазонах

Изменить **название окна**, можно передав строку в `figure` как параметр (рис. 1.36), либо с помощью метода `canvas.manager.set_window_title`.

```

1 # Способ 1
2 fig = p.figure('Название')
3
4 # Способ 2
5 fig.canvas.manager.set_window_title('Название')

```

Рис. 1.36. Задание названия окна

Размер окна в дюймах также указывается как параметр `figsize` метода `figure` (рис. 1.37).

```

1 fig = p.figure(figsize=(5, 2.5))

```

Рис. 1.37. Настройка размеров окна

Более сложные графики можно построить, дополнительно используя модуль NumPy (рис. 1.38). Подробнее о нем в разделе 1.4.

```

1 import matplotlib.pyplot as p
2 import numpy as np

```

Рис. 1.38. Подключение модулей

Для **генерации numpy-массивов** используются функции (рис. 1.39):

1. `arange` — принимает начало и конец требуемого диапазона, а также «шаг», то есть значение приращения (кол-во точек, то есть длина массива, вычисляется само);
2. `linspace` — также принимает начало и конец требуемого диапазона, а третьим параметром — количество равноудаленных точек в этом диапазоне (шаг подбирается автоматически).

```

1 a = np.arange(0, 100, 0.1)
2 b = np.linspace(0, 100, 1000)

```

Рис. 1.39. Два способа создать массив из 1000 значений от 0 до 100 с шагом 0.1

Если нужно задать значения прямо в коде, можно использовать функцию `np.array`, превращающую в numpy-массив обычный список или кортеж.

Массивы в NumPy заточены под скорость, а также под удобство выполнения операций над ними. Например, можно увеличить все элементы на значение простым «сложением массива и числа» (рис. 1.40). Подобным образом выполняются и прочие операции, например, умножение, вычисление

синуса или логарифма и т.д., что обеспечивает высокую удобность использования.

```
1 arr = np.arange(0, 5, 1) # [0 1 2 3 4]
2 arr = arr + 10           # [10 11 12 13 14]
```

Рис 1.40. Работа с массивами в numpy

Некоторые функции модуля:

- тригонометрические (sin, cos, tan, arccos, arcsin, arctan),
- квадратный корень (sqrt) и произвольная степень (power),
- модуль (abs),
- экспонента (exp) и логарифм (log, log2, log10),
- сумма (sum) и среднее значение (mean),
- максимальное (max) и минимальное (min) значения,
- медиана, дисперсия, среднеквадратическое отклонение...

Пример работы с ними представлен на рисунках 1.41 и 1.42.

```
1 x = np.arange(0.1, 4*np.pi+0.1, 0.1)
2 y = np.cos(x)
3 p.plot(x, y)
4 p.show()
```

Рис. 1.41. Код примера

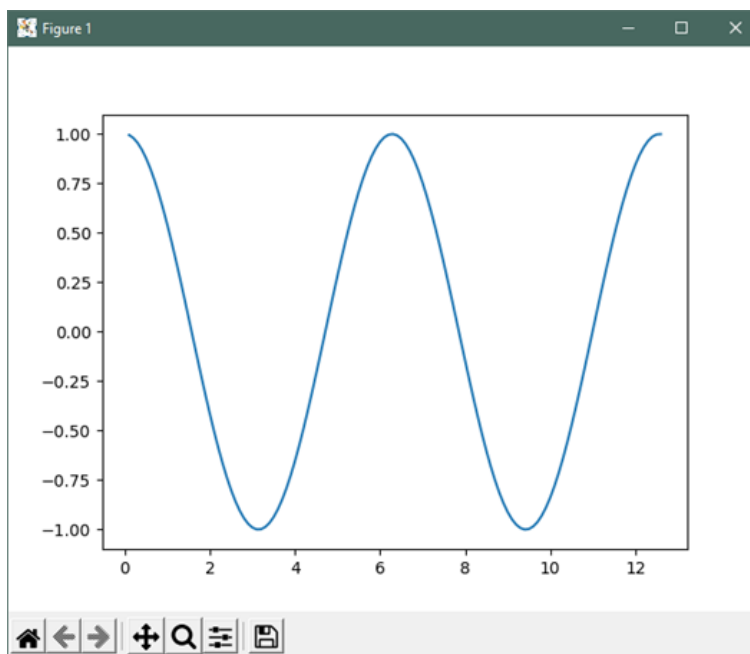


Рис. 1.42. График примера

Функция **scatter** строит точки, не соединяя их (рис. 1.43 и 1.44). При этом для нее справедливы многие настройки функции plot. Кроме того, можно указать параметр **marker** для указания как именно будут рисоваться точки (рис. 1.45).

```
1 import matplotlib.pyplot as p
2 import random as rnd
3
4 p.axes().set_facecolor('#333')
5
6 L = 25
7 x = range(L)
8 y = generate(x, 3) # Пользовательская функция генерации
9
10 p.scatter(x, y, color='lightcyan', marker='*')
11 p.show()
```

Рис. 1.43. Код построения дискретного набора точек

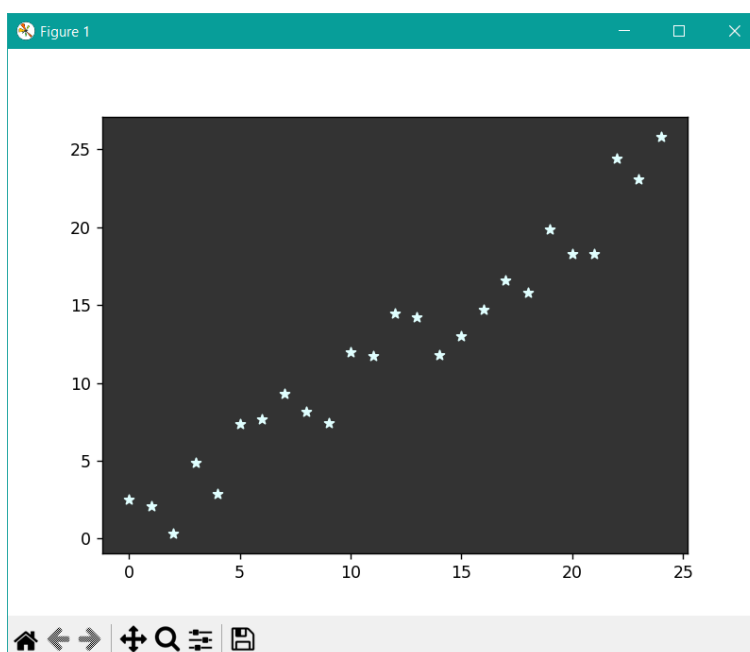


Рис. 1.44. График с точками в виде звезд

Добавление текста прямо на рисунок осуществляются функцией **text** с заданными координатами и строкой текста. Также можно указать необязательные параметры, например, размер, цвет и стиль текста, и многое другое (рис. 1.46 и 1.47).

"."	•	point	"p"	⬠	pentagon
","	.	pixel	"P"	⊕	plus (filled)
"o"	●	circle	"*"	★	star
"v"	▼	triangle_down	"h"	⬡	hexagon1
"^"	▲	triangle_up	"H"	⬢	hexagon2
"<"	◀	triangle_left	"+"	+	plus
">"	▶	triangle_right	"x"	×	x
"1"	⌵	tri_down	"X"	⊗	x (filled)
"2"	⌶	tri_up	"D"	◆	diamond
"3"	↶	tri_left	"d"	◇	thin_diamond
"4"	↷	tri_right	" "		vline
"8"	⬢	octagon	"_"	—	hline
"s"	■	square			

Рис. 1.45. Список маркеров

```

1 p.text(1.1, 0.5, # Координаты
2 'cos(x)', fontsize=18, color='b', fontstyle='italic')

```

Рис. 1.46. Код для добавления текста к графику

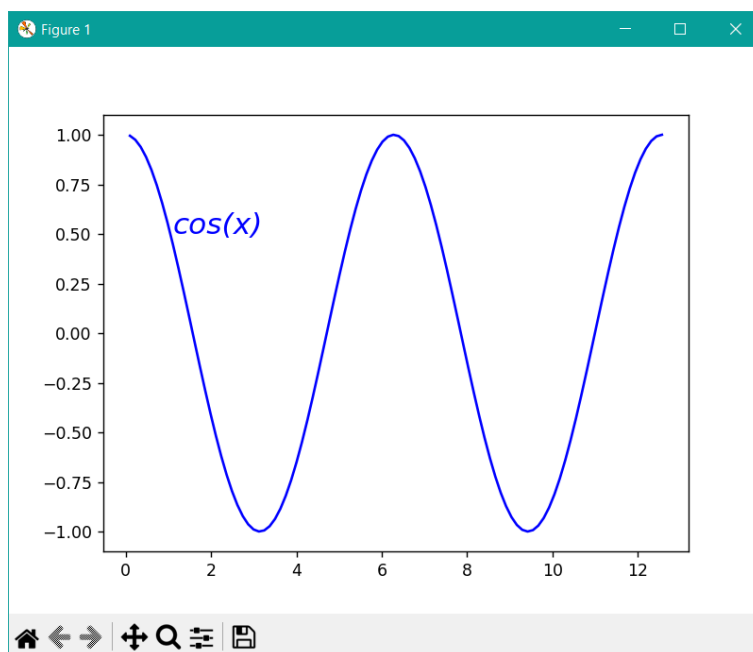


Рис. 1.47. График косинуса и текст к нему

Графики можно строить в полярной системе координат, указав параметр `polar` как `True` (рис. 1.48 и 1.49).

```
1 t = np.linspace(0, 6*np.pi, 300)
2 r = 2 * t
3
4 fig.add_subplot(polar=True)
5 p.plot(t, r, color='#345247')
```

Рис. 1.48. Код создания графика в полярных координатах

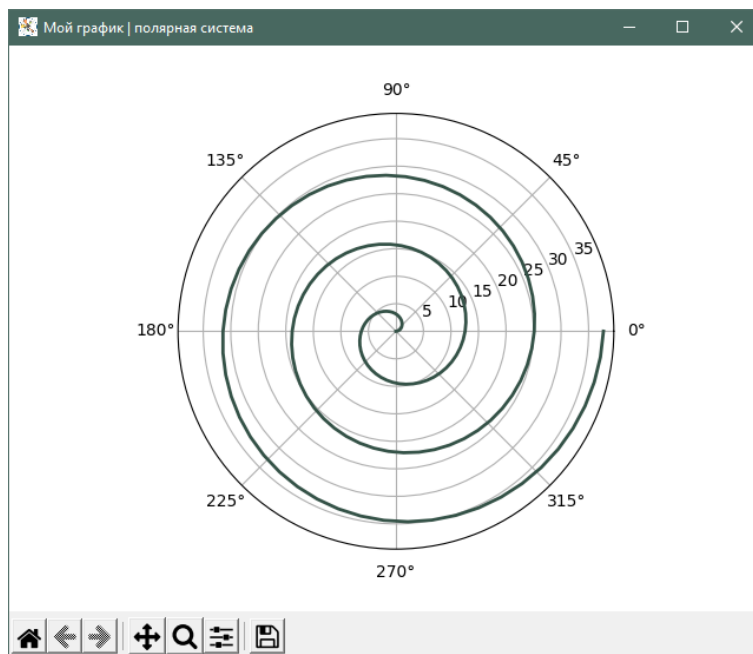


Рис. 1.49. График функции $r = 2 * \theta$

Трехмерные графики можно построить, указав параметр `projection='3d'` при создании координатной плоскости. На вход функции построения придется указать уже не 2 массива, а 3 матрицы, и «цветовую карту» вместо цвета (рис. 1.50 и 1.51).

```
1 import matplotlib.pyplot as p
2 import numpy as np
3
4 ax = p.figure().add_subplot(projection='3d')
5
6 # Задание матриц
7 u, v = np.mgrid[0:2*np.pi:40j, 0:np.pi:20j]
8 x = np.cos(u)*np.sin(v)
9 y = np.sin(u)*np.sin(v)
10 z = np.cos(v)
11
12 ax.plot_surface(x, y, z, cmap='autumn')
13 p.show()
```

Рис. 1.50. Код для трехмерного графика

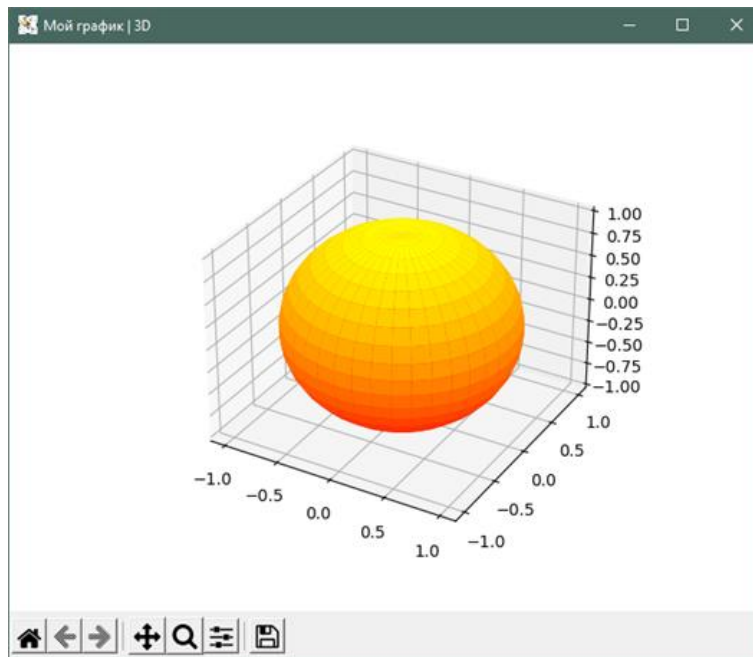


Рис. 1.51. График сферы по точкам

Геометрические объекты, например, окружность, круг или прямоугольник можно нарисовать, используя метод `gca` и `add_patch` (рис. 1.52 и 1.53).

```

1  c1 = p.Circle((0, 0), 3, color='r', fill=False)
2  r1 = p.Rectangle((-1, -1.5), 3.5, 4, color='g', fill=False)
3
4  ax = fig.add_subplot()
5  ax.set_xlim([-5, 5])
6  ax.set_ylim([-4, 4])
7  ax.grid(visible=True)
8
9  ax = p.gca()      # Конва (из текущих осей)
10 ax.add_patch(c1)  # Добавление круга
11 ax.add_patch(r1)  # Добавление прямоугольника
12 p.show()

```

Рис. 1.52. Код для построения объектов

Для построения **круговых диаграмм** (чартов) используется функция `pie` (рис. 1.54). Некоторые ее параметры:

- `x` — массив с размерами долей;
- `labels` — текстовые подписи;
- `radius` — радиус диаграммы;
- `startangle` — угол поворота в градусах против часовой стрелки (по умолчанию ноль направлен вдоль положительной оси OX);

- `rotatelabels` — вращать ли текст подписей;
- `autopct` — отображение чисел из `x` (выражением форматирования);
- `explode` — расстояние между долями;
- `shadow` — отображать ли тени;
- `legend` — создание легенды (ей также можно указать заголовок через параметр `title`).

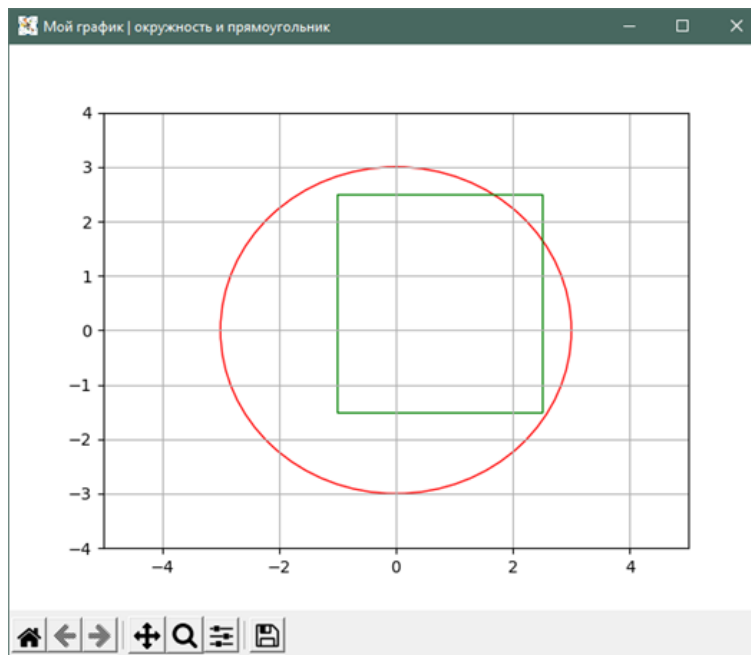


Рис. 1.53. Пустые квадрат и круг

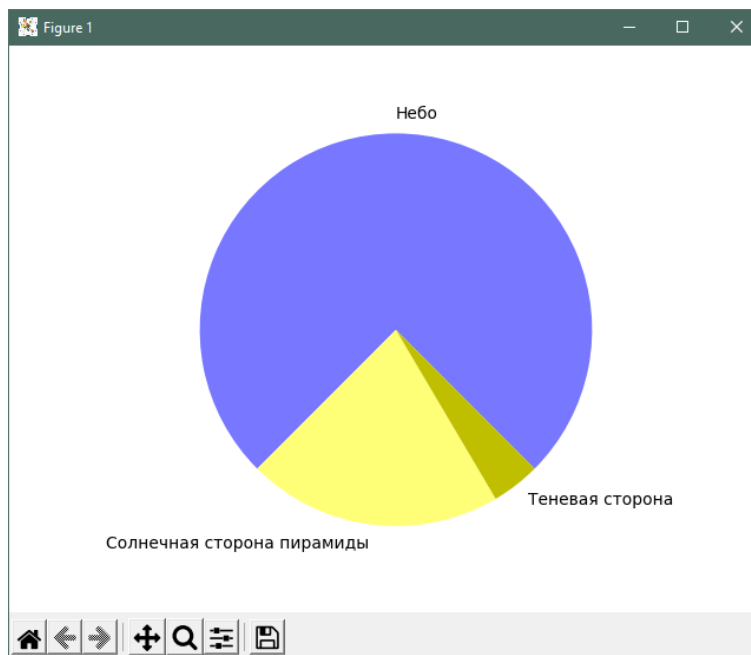


Рис. 1.54. Круговая диаграмма

Для построения **столбчатых диаграмм** используются функции `bar` (вертикальная) и `barh` (горизонтальная).

Некоторые параметры для `bar` (рис. 1.55):

- `x` — названия столбцов;
- `height` — высоты столбцов;
- `color` — цвета;
- `width` — толщина.

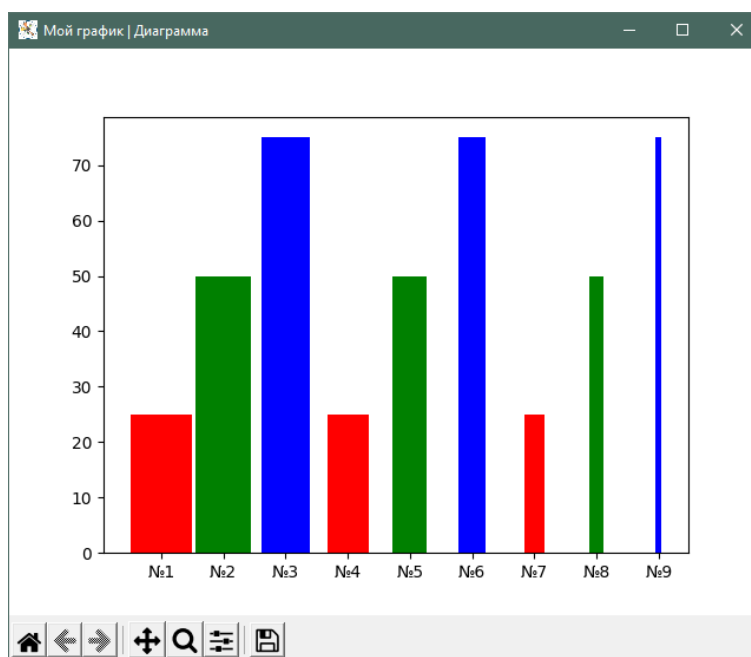


Рис. 1.55. Столбчатая диаграмма

1.4. Библиотека NumPy

Библиотека NumPy используется для научно-технических расчетов и высокопроизводительных вычислений. Она предоставляет доступ к оптимизированным математическим операциям над одномерными и многомерными массивами, средства для ввода и вывода данных и другие средства для проведения научных расчетов.

В пакете предоставляется новый тип данных — `ndarray` (n-мерный массив), над которым возможно выполнение векторных операций посредством одной команды (без явного использования циклов).

Для подключения модуля используется команда `import numpy as np`.

Следующие отличия от стандартных списков Python характерны массивам `ndarray`:

- массивы однородны (все элементы одного типа);
- массивы имеют фиксированный размер;
- математические операции над элементами массива чаще всего менее ресурсозатратны.

Информацию о массиве можно получить, используя **атрибуты** (рис. 1.56):

- `dtype` — ассоциированный тип (тип хранимых значений);
- `ndim` — размерность, то есть число измерений;
- `size` — длина, то есть суммарное число объектов по всем измерениям;
- `shape` — форма, то есть кортеж целых чисел, указывающих размер массива по каждому измерению;
- `itemsize` — размер в байтах одного элемента массива.

```
1  x = np.array([1, 2])
2
3  print(x.dtype)      # int64
4  print(x.ndim)       # 1
5  print(x.size)       # 2
6  print(x.shape)      # (2, )
7  print(x.itemsize)   # 8
```

Рис. 1.56. Атрибуты `numpy`-массива

Для создания массивов заданной длины, заполненных нулями или единицами, используются методы `zeros` и `ones` (рис. 1.57).

```
1  z = np.zeros(shape=(5,), dtype=float)
2  # [0. 0. 0. 0. 0.]
3
4  o = np.ones(shape=(3,2), dtype=int)
5  # [[1 1]
6  #  [1 1]
7  #  [1 1]]
```

Рис. 1.57. Инициализация массива нулями и матрицы единицами

Арифметические, побитовые операции и операции сравнения применяются к массивам поэлементно (рис. 1.58).

```

1  x = np.array([1, 2, 3])
2  y = np.array([4, 5, 6])
3
4  print(x + 10)    # [11 12 13]
5  print(x + y)     # [5 7 9]
6  print(x >= 2)    # [False True True]
7  print(x | 2)     # [3 2 3]
8  print(x << 2)    # [ 4  8 12]

```

Рис. 1.58. Операции над массивами

Вычисление **минимума** и **максимума** стандартными функциями Python возможны только для одномерного массива. Для произвольного числа измерений используются функции `np.maximum` и `np.minimum`.

Также поддерживаются следующие унарные функции над массивами: `sqrt`, `exp`, `sin`, `cos`, `log`, `log2` и бинарные функции `add`, `subtract`, `multiply`.

Матричное произведение вычисляется методом `dot` или оператором `@` (рис. 1.59).

```

1  x = np.array([[4], [5]])
2  y = np.array([[1, 2, 3]])
3
4  print(x @ y)
5  # [[4 8 12]
6  #  [5 10 15]]

```

Рис. 1.59. Матричное произведение

Для матриц определен метод **транспонирования** «.T» (рис. 1.60).

```

1  x = np.array([[1, 2, 3], [4, 5, 6]])
2
3  print(x.T)
4  # [[1 4]
5  #  [2 5]
6  #  [3 6]]

```

Рис. 1.60. Транспонирование матрицы

Следующие методы позволяют вычислить:

- `trace` — след матрицы;
- `linalg.inv` — обратную матрицу;
- `linalg.det` — определитель матрицы;
- `linalg.matrix_rank` — ранг матрицы;
- `linalg.matrix_power` — возведение в целочисленную степень;

- `linalg.eigvals` — собственные значения матрицы;
- `linalg.solve(A, B)` — решение системы линейных уравнений $Ax = B$.

Срезы вычисляются по каждой оси отдельно (рис. 1.61).

```

1 x = np.array([[1, 2, 3, 4],
2               [5, 6, 7, 8],
3               [9, 10, 11, 12]])
4
5 print(x[1:, 2:4])

```

Рис. 1.61. Получение среза матрицы

В результате будет напечатан срез с первой строки до конца и со второго по четвертый (не включительно) столбец.

1.5. Пакет SciPy

Данный пакет предназначен для проведения сложных научно-технических расчетов и содержит модули для решения задач линейной алгебры, анализа сигналов и обработки изображений.

И NumPy, и SciPy являются библиотеками Python, которые используются для математического и числового анализов. NumPy содержит данные массивов и операции, такие как сортировка, индексация, а SciPy состоит из числового кода. И хотя в NumPy есть функции для работы с линейной алгеброй, преобразованиями Фурье и прочие, в SciPy они представлены в полном виде вместе с массой других. А для полноценного научного анализа в Python нужно устанавливать и NumPy, и SciPy, поскольку последняя построена на базе NumPy.

В SciPy есть набор пакетов для разных научных вычислений:

- `cluster` — алгоритмы кластерного анализа;
- `constants` — физические и математические константы;
- `fftpack` — быстрое преобразование Фурье;
- `integrate` — решения интегральных и обычных дифференциальных уравнений;
- `interpolate` — интерполяция и сглаживание сплайнов;
- `io` — ввод и вывод;

- linalg — линейная алгебра;
- ndimage — n-размерная обработка изображений;
- odr — метод ортогональных расстояний;
- optimize — оптимизация и численное решение уравнений;
- signal — обработка сигналов;
- sparse — разреженные матрицы;
- spatial — разреженные структуры данных и алгоритмы;
- special — специальные функции;
- stats — статистические распределения и функции.

2. Задачи на matplotlib

2.1. Примеры

Задача 1. Построить график функции $y = \sqrt[3]{x^2} \pm \sqrt{1-x^2}$ (рис. 2.1 и 2.2).

```

1  import matplotlib.pyplot as p
2  import numpy as np
3
4  def sqr(x):
5      return np.power(x, 2)
6
7  # Название окна
8  p.figure('Черви')
9
10 # Генерация массива "по X"
11 x = np.linspace(-1, 1, 200)
12
13 # Массивы Y
14 y1 = np.power(sqr(x), 1/3) + np.sqrt(1-sqr(x))
15 y2 = np.power(sqr(x), 1/3) - np.sqrt(1-sqr(x))
16
17 # Отрисовка
18 p.plot(x, y1, linewidth=5, color='r')
19 p.plot(x, y2, linewidth=5, color='r')
20 p.show()
```

Рис. 2.1. Код решения задачи 1

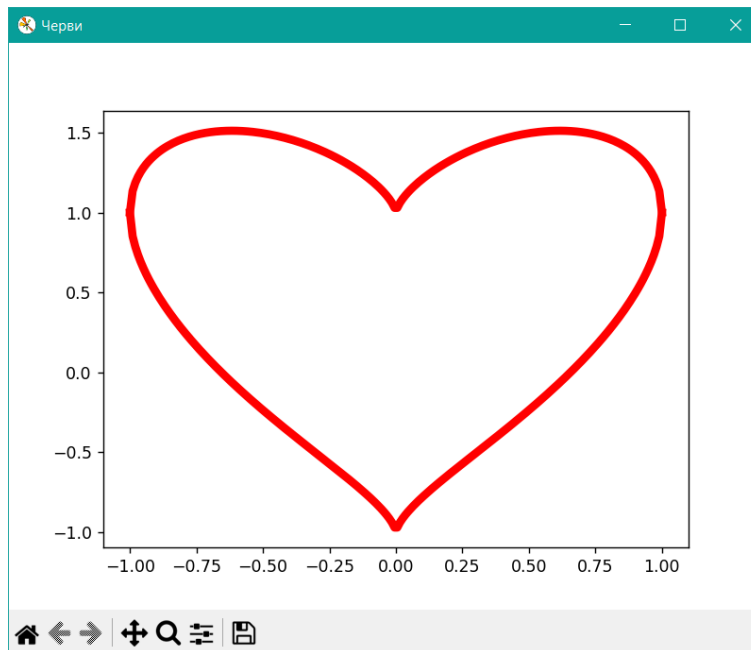


Рис. 2.2. График функции $y = \sqrt[3]{x^2} \pm \sqrt{1 - x^2}$

В строках 1-2 происходит подключение модулей.

В строках 4-5 объявлена функция `sqr`. Она не является обязательной в данной задаче и нужна только для того, чтобы не писать каждый раз `pr.power(x, 2)`.

В строке 8 задается название окна.

В строках 11-15 вычисляются значения массивов координат точек, по которым будет построен график.

В строках 18-19 вызывается метод построения графиков. Верхняя часть «сердца» в 18ой строке и нижняя — в 19ой.

Задача 2. Построить в полярных координатах график функции $r = 2 * \theta$ (рис. 2.3 и 2.4).

Основное отличие от предыдущего примера — строка 5, в которой указывается параметр `polar=True`.

```

1 import matplotlib.pyplot as p
2 import numpy as np
3
4 fig = p.figure('Полярная система координат')
5 fig.add_subplot(polar=True)
6
7 laps = 3 # Кругов
8
9 t = np.linspace(0, laps*2*np.pi, 300)
10 r = t*2
11
12 p.plot(t, r, color='b', linewidth=2)
13 p.show()

```

Рис. 2.3. Код решения задачи 2

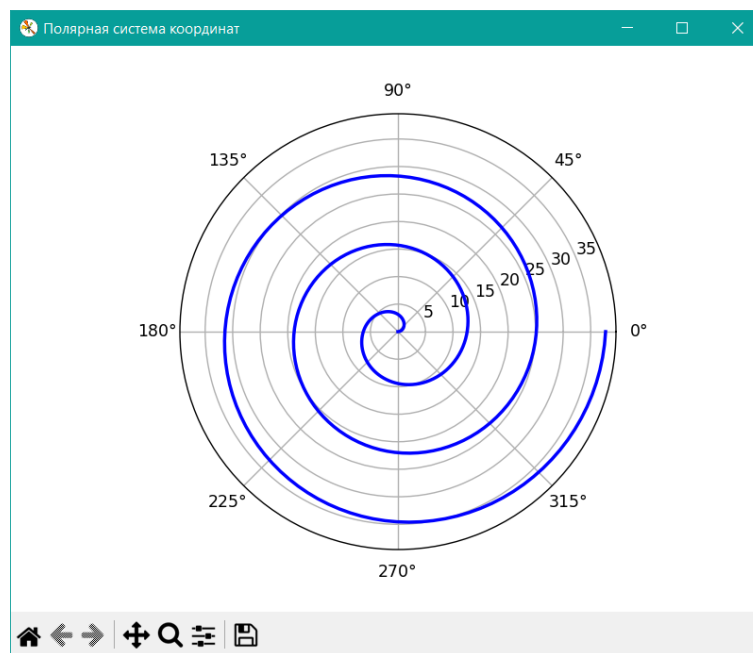


Рис. 2.4. График функции $r = 2 * \theta$

Задача 3. Построить горизонтальную столбчатую диаграмму с 7 случайными целочисленными значениями из интервала $[1; 100]$. Столбцы (то есть «полосы») раскрасить в цвета радуги (рис. 2.5 и 2.6).

В строках 7-10 происходит генерация списка из случайных чисел.

В строках 12-18 настраивается диаграмма. Здесь width это размеры (длины) полос, height это кортеж «толщин» полос. Значение 1 соответствует 100% предоставляемой ширины (то есть полосы будут впритык друг к другу), в примере все полосы одинаковой ширины 70%.


```

1  import matplotlib.pyplot as p
2  from random import randint
3
4  fig = p.figure('BARH')
5  ax = fig.subplots()
6
7  size = 7
8  values = [0] * size
9  for i in range(size):
10     values[i] = randint(1, 100)
11
12  ax.barh(
13     width = values,
14     height = (0.7,)*size,
15     y      = ['1', '2', '3', '4', '5', '6', '7'],
16     color  = ['r', 'orange', 'yellow', 'g',
17              'skyblue', 'b', 'darkorchid']
18  )
19
20  p.show()

```

Рис. 2.5. Код решения задачи 3

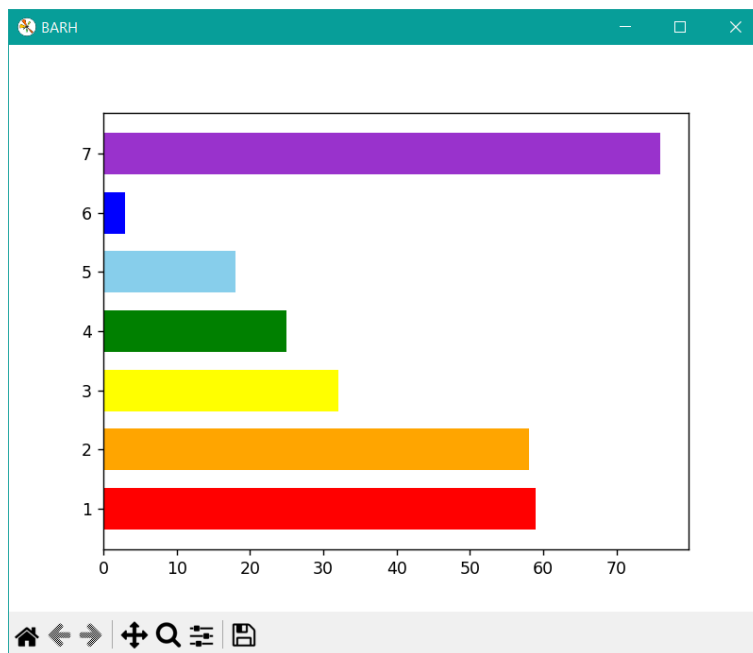


Рис. 2.6. Столбчатая горизонтальная диаграмма

Список `y` — это набор текстовых подписей к каждой полосе. В данном примере они таким образом пронумерованы. Цвета полос задаются списком `color`. Все данные должны быть последовательностями одинакового размера.

2.2. Условия задач

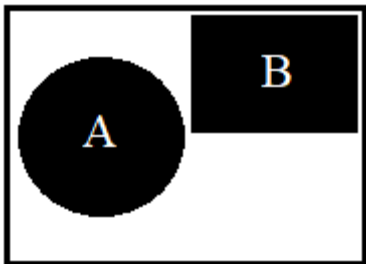
Задача должна быть выполнена на языке Python с использованием библиотек `matplotlib`, `numpy`, и, при необходимости, `math` и `random`. Условия задачи и автор должны быть указаны в комментариях.

Все графики строятся в одном окне так, как представлено на схемах. Название окна должно быть осмысленным и не должно оставаться по умолчанию. Область рисования должна быть настроена так, чтобы графики отображались корректно (*то есть, например, обе ветки гиперболы должны быть видны, или окружность не должна «сплющиваться»*).

В задании указаны формулы, по которым должен быть построен графики. Графики должны быть наглядными. *То есть не кусочек, по которому ничего не понятно. Каждый вариант выполняет все (заглавные) буквы. Маленькая буква в номере задачи относится к стилям (см. конец файла).* Если под одной буквой несколько формул — все их графики строятся в одних осях (на одном рисунке, вместе, одного цвета и стиля).

Кружок на схеме означает полярные координаты, прямоугольник — декартовы.

1



$$(A) \ r = \sin \frac{111\varphi}{122}$$

(B) Формулы заданы параметрически:

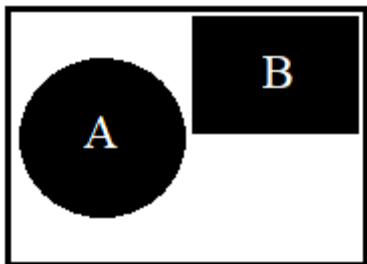
$$x = a \cdot \cos(t), \ y = a \cdot \sin(2 \cdot t)/2$$

и

$$x = b \cdot \cos(t), \ y = b \cdot \sin(2 \cdot t)/2$$

где a и b — случайные вещественные числа из отрезка $[1, 5]$

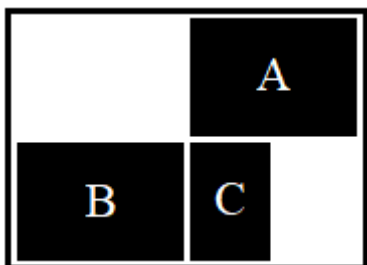
2



(A) $y = \pm\sqrt{\sqrt{1+4x^2} - x^2 - 1}$

(B) $r = 100(1.2 + 0.9 \cos(1.8\varphi) + 0.1 \cos(0.9\varphi))$

3



(A) Формула задана параметрически:

$$x = 6 \cos t + \cos 6t$$

$$y = 6 \sin t - \sin 6t, \text{ где } t \in [0; 2\pi]$$

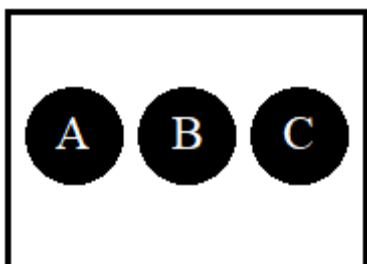
(B) $y = e^{\frac{-x}{15}} \sin x$

(C) Ломанную. Координаты точек:

(1, a), (2, b), (3, c), (4, d), (5, f),

где a, b, c, d, f — случайные вещественные числа из отрезка $[-10, 10]$

4



(A)

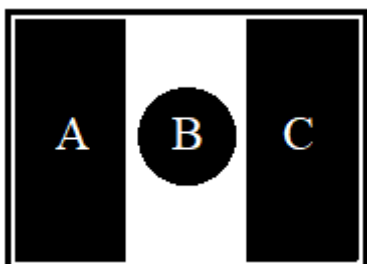
$$r = \theta, \text{ при } \theta \geq 0$$

$$r = -\theta - \pi, \text{ при } \theta \leq -\pi$$

(B) $r = \sin(11 \cdot \theta)$

(C) $r = \cos \frac{4\theta}{7}$

5



(A) Формула задана параметрически:

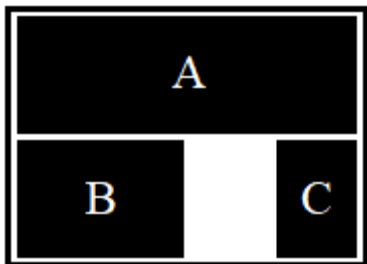
$$x = 9 \cos t + \cos 9t$$

$$y = 9 \sin t - \sin 9t, \text{ где } t \in [0; 2\pi]$$

(B) $r = \sqrt[{\pm 2.5}]{\cos(\pm 2.5\varphi)}$

(C) $y = a/x$, где a — случайное число из $[-5; 0) \cup (0; 5]$

6



(A) $y = \ln(x)$

(B) Формула задана параметрически:

$$x = 3 \cos t + 5 \cos \frac{3t}{2}$$

$$y = 3 \sin t - 5 \sin \frac{3t}{2}$$

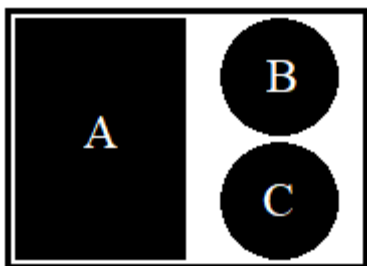
где $t \in [0; 4\pi]$

(C) Треугольник. Координаты вершин:

$(-2, a), (0, b), (2, c),$

где a, b, c — случайные вещественные числа из отрезка $[-10, 10]$

7



(A) Формула задана параметрически:

$$x = 2 \cos t + 4 \cos \frac{2t}{3}$$

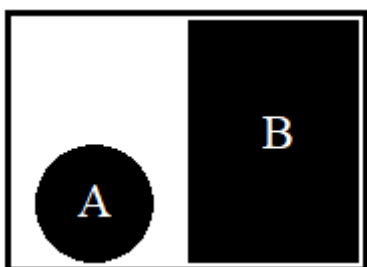
$$y = 2 \sin t - 4 \sin \frac{2t}{3}$$

где $t \in [0; 6\pi]$

(B) $r = {}^{\pm 1.25}\sqrt{\sin(\pm 1.25\varphi)}$

(C) $r = a$, где a — случайное из $[1, 10]$

8



(A) Ломаная

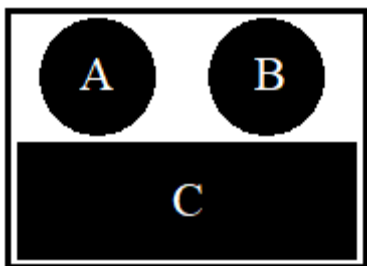
$$\varphi = \frac{k\pi}{12}$$

$$r = \begin{cases} 10, & \text{если } k \text{ четная} \\ 3, & \text{если } k \text{ нечетная} \end{cases}$$

$$k = \overline{0, 24}$$

$$(B) y = \begin{cases} \sqrt{1-x^2}, & \text{при } x \in [-1, 1) \\ -\sqrt{1-(x-2)^2}, & \text{при } x \in [1, 3) \\ \sqrt{1-(x-4)^2}, & \text{при } x \in [3, 5] \end{cases}$$

9



(A) Ломаная

$$\varphi = \frac{k\pi}{3}, \text{ где } k = \overline{0, 6}$$

$$r = a, \text{ где } a \text{ — случайное из } [1, 10]$$

$$(B) r = 1 + 7 \cos 5\varphi + 4 \sin^2 5\varphi + 3 \sin^4 5\varphi$$

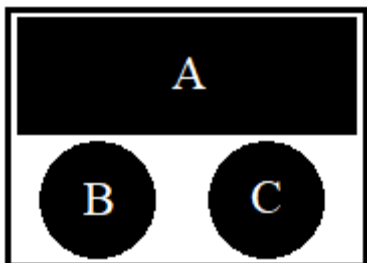
(C) Формула задана параметрически:

$$x = 3 \cos t + 2 \cos 3t$$

$$y = 3 \sin t - 2 \sin 3t$$

$$\text{где } t \in [0; 2\pi]$$

10



(A) Формула задана параметрически:

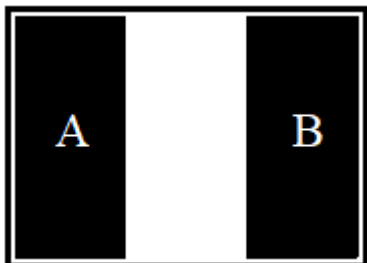
$$x = \sin 5t$$

$$y = \sin 4t$$

$$(B) r = \frac{3 \cos \varphi \sin \varphi}{\cos^3 \varphi + \sin^3 \varphi}$$

$$(C) r = 2 - 0.5 \sin 50\varphi + \cos 7\varphi$$

11



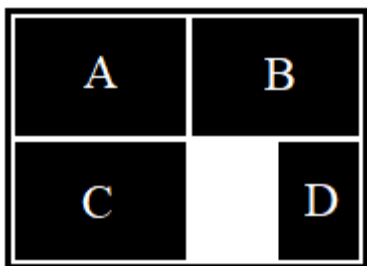
$$(A) |y| = \frac{x\sqrt{2+4x+2\sqrt{1-4x}}}{1+\sqrt{1-4x}}, \text{ где } x \in [-2; 0.25]$$

(B) Формула задана параметрически:

$$x = \sin 9t$$

$$y = \sin 8t$$

12

**(A)** Формула задана параметрически:

$$x = 5 \cos t + 3 \cos 5t$$

$$y = 5 \sin t - 3 \sin 5t$$

где $t \in [0; 2\pi]$ **(B)** Формула задана параметрически:

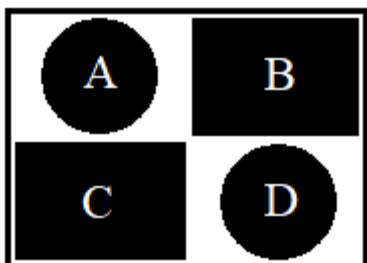
$$x = e^{\sqrt{t}} \cos t$$

$$y = e^{\sqrt{t}} \sin t$$

$$\text{(C)} \quad y = \pm \sqrt{x^3 + x^2}$$

$$\text{(D)} \quad y = x^3$$

13



$$\text{(A)} \quad r = \sqrt[{\pm 1.5}]{\cos(\pm 1.5\varphi)}$$

$$\text{(B)} \quad y = x^5 - x^3$$

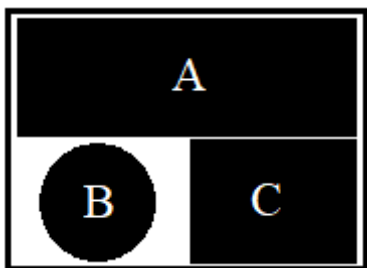
$$\text{(C)} \quad y = 2^{\frac{x}{10}} \cos x$$

(D) Ломаная

$$\varphi = \frac{2k\pi}{5}, \text{ где } k = \overline{0, 10}$$

$$r = a, \text{ где } a \text{ — случайное из } [1, 10]$$

14

**(A)** Сгенерировать случайную

последовательность из 30 элементов

по следующему принципу: каждое следующее

значение получается из предыдущего +

случайное число из $[-10, 10]$. Нарисовать

дискретный график этой последовательности.

В качестве маркеров использовать

шестиугольники.

(B) Пять линий, заданных формулой:

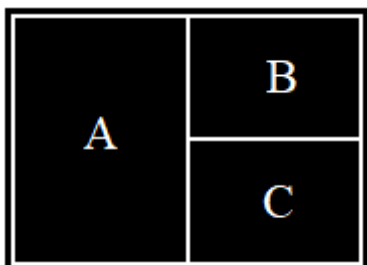
$$r = \cos \theta + k,$$

где $k = \overline{0, 4}$ **(C)** Параметрически заданная функция:

$$x = \operatorname{sgn}(t) \sqrt{|t|} \cos t$$

$$x = \sqrt{|t|} \sin t$$

15

**(A)** Построить вертикальную столбчатую

диаграмму для 7 значений функции

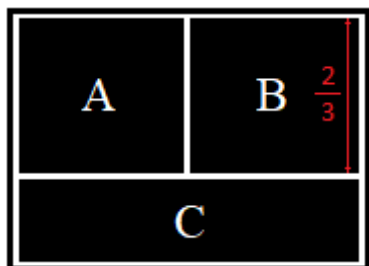
$$y = ae^{-\frac{(x-b)^2}{2c^2}} \text{ в точках } 0, 1, 2, 3, 4, 5 \text{ и } 6, \text{ где:}$$

 a — случайное вещественное из $[1; 5]$, $b = 3$, c — случайное вещественное из $[1; 2]$.**(B)** Построить «обычный» график функциииз пункта А с теми же значениями a , b и c .**(C)** Параметрически заданная функция:

$$x = \cos^3 t$$

$$x = \sin^3 t$$

16

**(A)** Параметрически заданная функция:

$$x = 4(\cos t + \frac{5}{4}\cos(\frac{4}{5}t))$$

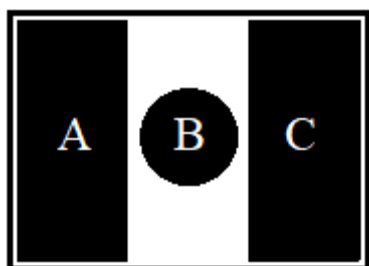
$$y = 4(\sin t - \frac{5}{4}\sin(\frac{4}{5}t))$$

$$\text{(B)} \quad y = \pm\sqrt{x^2 - 1}$$

(C) Построить горизонтальную столбчатую диаграмму с 3 случайными целочисленными значениями из диапазона [1, 100].

Цвета столбцов также должны быть случайными (разными при каждом запуске).

17

**(A)** Параметрически заданная функция:

$$x = 7(\cos t + \frac{2}{7}\cos(\frac{7}{2}t))$$

$$y = 7(\sin t - \frac{2}{7}\sin(\frac{7}{2}t))$$

(B) Круговая диаграмма, показывающая соотношение ваших оценок в последнюю сессию. Учитываются экзамены, зачеты с оценкой и практика. Каждая доля должна быть своего цвета, подписана и указано значение-процент среди всех оценок.

(C) Параметрически заданная функция:

$$x = \frac{u^2 - 1}{u^2 + 1}$$

$$y = u \frac{u^2 - 1}{u^2 + 1}$$

где $u = \text{tg}(t)$.

Стили и дополнительное задание. Графики должны быть раскрашены в них в том же порядке, как указано здесь (если в варианте графиков меньше чем цветов/стилей, использовать первые из указанных):

- a. Цвет: любой который можно задать одной буквой (но для каждого свой).
Линия: сплошная, пунктирная, сплошная, пунктирная.
Толщина: 2, 3, 2, 3.
Сетка: ☒, ☒, ☒, ☒.
- Задать пользовательские значения вдоль оси x (θ или φ).
- b. Цвет: различные оттенки серого, заданные как html коды.
Линия: все выполнить «штрих-пунктиром».
Толщина: 2, 2, 2, 2.
Сетка: ☒, ☒, ☒, ☒.
- Цвет сетки для всех графиков — lightskyblue.
- c. Цвет: forestgreen, lime, mediumvioletred, peru.
Линия: сплошная, штриховая, сплошная, сплошная.
Толщина: 1, 2, 1, 1.
Сетка: ☒, ☒, ☒, ☒.
- Задать пользовательские значения вдоль оси y (r).
- d. Цвет: teal, coral, sandybrown, palegreen.
Линия: штриховая, сплошная, штриховая, штриховая.
Толщина: 2, 1, 2, 2.
Сетка: ☒, ☒, ☒, ☒.
- Сетку сделать пунктирной.
- e. Цвет: #adff2f, #4b0082, #b8860b, #ffefd5.
Линия: штриховая, пунктирная, сплошная, штрих-пунктир.
Толщина: 2, 3, 1, 2.
Сетка: ☒, ☒, ☒, ☒.
- На первом графике (то есть внутри осей) подписать свою фамилию.
- f. Цвет: lightgoldenrodyellow.
Линия: сплошная, штриховая, сплошная, штриховая.
Толщина: 5, 3, 7, 13.
Сетка: нет.
Цвет координатной плоскости: #333.
Цвет окна: #8e9ca3.

3. Декораторы

3.1. Написание пользовательского декоратора

Декораторы — это обертки вокруг Python-функций (или классов), которые изменяют работу того, к чему они применяются.

Декораторы используются, чтобы изменять работу существующих функций или классов, добавлять новые возможности и обеспечивать безопасность.

Функции в Python являются объектами, соответственно, их можно возвращать из другой функции или передавать в качестве аргумента. Также следует помнить, что функция может быть определена и внутри другой функции.

На рисунке 3.1 представлен пример написания декоратора с комментариями. Декоратор создается для любой функции, принимающей один параметр и возвращающей строку.

```
1  # Декоратор, который добавляет скобки к функции,  
2  # возвращающей строку  
3  
4  # Параметр func — это та функция, которая будет декорирована  
5  def add_brackets_decorator(func):  
6      # Внутри нужно определить функцию-обертку wrapper  
7      # Она будет подставляться вместо декорируемой  
8      # Она вызывает внутри исходную функцию  
9      def wrapper(x):  
10         # Именно здесь добавляются скобки  
11         return '[' + func(x) + ']'  
12  
13     # В конце обязательно нужно вернуть эту функцию  
14     # созданную внутри декоратора  
15     return wrapper
```

Рис. 3.1. Создание пользовательского декоратора

В строке 5 объявлен декоратор, который, с точки зрения кода, является функцией. Ему дано имя, которое будет использовано в дальнейшем, при декорировании конкретной функции. Параметром `func` декоратора является «обрабатываемая» функция. Имя параметра может быть любым. По этому имени функция будет вызываться внутри декоратора (строка 11).

В строке 9 объявлена вложенная в декоратор функция. Это важно, так как в результате декорирования именно она будет вызываться вместо исходной. Декоратор — это механизм, который позволяет «подменить» исходную функцию созданной.

Обычно вложенной функции дается название `wraper`, но это не существенно, так как ее имя используется только внутри декоратора.

Она будет вызывать в себе исходную (строка 11). Это обязательно чтобы основная логика функции, которая декорируется (исходная), срабатывала. Значение `x` передается из параметра `wraper` в декорируемую функцию `func`. Также здесь вносится и дополнительная логика: добавление скобок.

В конце обязательно вернуть созданную функцию, чтобы подмена работала.

3.2. Применение декоратора к функции

Есть некоторая функция (рис. 3.2).

```
1  # Обычная функция
2  def to_string_upper(x):
3      return str(x).upper()
```

Рис. 3.2. Функция

Она принимает один аргумент и возвращает строку. То есть подходит для предыдущего примера. Чтобы применить к ней декоратор, используется синтаксис **@имя_декоратора** перед объявлением функции (рис. 3.3).

```
1  @add_brackets_decorator
2  def to_string_upper(x):
3      return str(x).upper()
```

Рис. 3.3. Функция с декоратором

Теперь не важно, как написана функция: как на рисунке 3.2 или как на рисунке 3.3 — использование этой функции не изменится (рис. 3.4). Однако, в зависимости от того, указан ли был декоратор, изменится поведение. То есть добавление декоратора не требует модификации внешнего кода, в чем заключается одно из преимуществ декораторов.

```
1 print(to_string_upper('Текст'))
```

Рис. 3.4. Вызов функции

В случае, когда декоратор не применялся к функции (рис. 3.2), будет напечатано «ТЕКСТ», если функция была декорирована (рис. 3.3), будет напечатано «[ТЕКСТ]».

Обертка функции без @. Поскольку по своей сути декоратор — это функция, которая принимает в качестве параметров одну функцию, а возвращает другую, функцию можно декорировать вручную как показано на рисунке 3.5.

```
1 to_string_upper = add_brackets_decorator(to_string_upper)
```

Рис. 3.5. Подмена функций

Результат будет аналогичным тому что делала строка 1 на рисунке 3.3.

Можно использовать **несколько декораторов для одной функции** (рис. 3.6).

```
1 def add_brackets_decorator(func):
2     def wrapper(x):
3         return '[' + func(x) + ']'
4     return wrapper
5
6 def double_decorator(func):
7     def wrapper(x):
8         return func(x) * 2
9     return wrapper
10
11 @double_decorator
12 @add_brackets_decorator
13 def to_string_upper(x):
14     return str(x).upper()
15
16 print(to_string_upper('Текст'))
```

Рис. 3.6. Два декоратора

При этом важен порядок декорирования. Сначала применяется декоратор, который «ближе к функции», то есть объявленный непосредственно над ее именем. Затем выполняется следующий над ним и так далее. Результат выполнения данного примера: «[ТЕКСТ][ТЕКСТ]». Другой порядок указания декораторов (рис. 3.7) даст результат «[ТЕКСТТЕКСТ]».

```

1  @add_brackets_decorator
2  @double_decorator
3  def to_string_upper(x):
4      return str(x).upper()
5
6  print(to_string_upper('Текст'))

```

Рис. 3.7. Два декоратора в другом порядке

Один декоратор с несколькими функциями. Один и тот же декоратор может использоваться с различными функциями, тем самым иногда позволяя избегать дублирования кода. Это еще одно преимущество декораторов.

Свойства и особенности декораторов:

- декораторы несколько замедляют вызов функции;
- невозможно «раздекорировать» функцию — если функция декорирована это не отменить;
- декораторы могут затруднить отладку, так как не ясно где возникла ошибка: в функции или в декораторе;
- декораторы могут быть использованы для расширения возможностей функций из сторонних библиотек код которых программист не может изменять.

3.3. Переменное число аргументов функции

Чтобы написать функцию с переменным числом аргументов используется синтаксис «*args, **kwargs», где *args — это сокращение от arguments и представляет из себя список позиционных аргументов, а **kwargs — сокращение от keyword arguments и является списком именованных аргументов.

Это полезно для создания декораторов, которые не зависят от того, сколько и какие параметры имеет декорируемая функция (рис. 3.8). С помощью такого синтаксиса из декоратора просто берутся все аргументы и передаются в оборачиваемую функцию, вызывающуюся внутри декоратора (строка 5).

```

1  def perf_time(func):
2      import time as T
3      def wrapper(*args, **kwargs):
4          t = T.time()
5          res = func(*args, **kwargs)
6          print(['[' + func.__name__ + ']: %0.8f' %
7                (T.time() - t)])
8          return res
9      return wrapper

```

Рис. 3.8. Декоратор, печатающий время выполнения функции

Данный декоратор можно применять для функции с любым числом аргументов, включая ранее рассмотренный пример, где объявлялась функция с одним аргументом.

Это справедливо и для функции без параметров.

Также данный декоратор получает результат декорируемой функции в строке 5, ничего с ним не делает и возвращает в качестве результата в строке 8. Поэтому им можно декорировать функцию с любым типом возвращаемого значения, включая функции без return (на самом деле они возвращают None).

3.4. Декораторы с параметрами

В декоратор можно передавать аргументы. Однако декоратор должен принять функцию в качестве первого аргумента, поэтому потребуется дополнительный уровень вложенности.

В примере на рисунке 3.9. декоратор, повторяющий функцию N раз.

```

1  def repeat(N):
2      def repeat_dec(func):
3          def wrapper(*args, **kwargs):
4              for i in range(N):
5                  func(*args, **kwargs)
6              return wrapper
7          return repeat_dec
8
9      @repeat(3)
10     def hello_world():
11         print('Hello World')
12
13     hello_world()

```

Рис. 3.9. Декоратор с параметрами

На самом деле это не совсем декоратор, а скорее функция, возвращающая декоратор.

В результате при любом вызове функции `hello_world`, строка «Hello World» будет печататься трижды, так как в строке 9 в декоратор передано значение «3».

3.5. Декораторы классов

Декоратор класса принимает на вход класс и возвращает модифицированный класс. Общий синтаксис представлен на рисунке 3.10.

```
1 def MyClassDec(cls):
2     class Wrapper:
3         # Поля и методы
4         ...
5     return Wrapper
6
7 @MyClassDec # класс декорируется
8 class SomeClass:
9     # Поля и методы оригинального класса
10    ...
11
12 # Реально будет создан экземпляр подмененного класса
13 x = SomeClass()
```

Рис. 3.10. Схема декоратора класса

4. ООП в Python

4.1. Объектно-ориентированное программирование

Объектно-ориентированное программирование используется для того, чтобы связать поведение сущности с её данными и спроецировать объекты реального мира и бизнес-процессов в программный код.

Класс (тип) — это описание того, какими свойствами и поведением будет обладать объект. А объект (инстанс) — это экземпляр с собственным состоянием этих свойств.

Три основных принципа ООП:

1. Наследование — это механизм системы, который позволяет получать одними классами свойства и поведение других классов для дальнейшего

расширения или модификации, что позволяет избегать лишней работы, дублирования кода и дублирования логики.

Также механизм наследования позволяет создавать общий каркас для разных классов, например, квадрат, круг и треугольник — фигуры на плоскости, для каждой из которых может быть вычислена ее периметр и площадь. Для многих решений может оказаться удобным сделать эти три класса производными от общего родительского класса-фигуры с методами нахождения площади и периметра. То есть выделить общие черты (получая тем самым еще один важный принцип — абстракцию).

2. Полиморфизм — свойство системы, позволяющее иметь множество реализаций одного интерфейса. То есть внешний код ничего не знает о реализации вызываемого метода.

Полиморфизм дословно означает «множество форм» — вызывающий код видит только название функции, не зная и не заботясь о том, какую конкретно форму (тело функции) она принимает. Например, метод вычисления площади для квадрата и круга может выглядеть одинаково, однако квадрат вычисляет площадь как «длина стороны во второй степени», а круг — « πr^2 ».

Вместе с наследованием позволяет изменять поведение методов в дочерних классах.

3. Инкапсуляция — это контроль доступа к полям и методам объекта. Во многих языках частью инкапсуляции является сокрытие данных (модификаторы доступа). Но под контролем доступа подразумевается не только можно получить доступ или нет, но и различные валидации, подгрузки, вычисления и другое динамическое поведение.

Подлинное назначение инкапсуляции — собрать в одном месте знания, относящиеся к устройству некой сущности, правилам обращения и операциям с ней (связать поведение сущности с ее данными).

Инкапсуляция появилась гораздо раньше и существует не только в ООП. Так, например, модули в программах на С — это тоже инкапсуляция.

4.2. Написание пользовательского класса

Для создания нового класса используется ключевое слово `class` после которого указывается его имя (рис. 4.1). После этого можно перечислить его поля. Все поля должны быть проинициализированы (присвоено значение).

```
1 class Point2d:
2     'Точка на плоскости'
3     x = 0
4     y = 0
```

Рис. 4.1. Объявление класса «Точка»

На самом деле, в отличие от других языков программирования, в Python перечислять поля «в теле класса» необязательно. Достаточно присвоить им значения в *инициализаторе* (см. раздел 4.3).

Создание экземпляра происходит по имени класса (рис. 4.2).

```
1 p1 = Point2d()
```

Рис. 4.2. Создание экземпляра

Методы — это функции внутри класса. Отличие состоит в обязательном параметре `self` — ссылке на объект, в контексте которого вызывается метод (рис. 4.3).

```
1 class Point2d:
2     'Точка на плоскости'
3     x = 0
4     y = 0
5
6     # Расстояние между точками
7     def distance(self, other):
8         return ((self.x - other.x)**2 +
9                 (self.y - other.y)**2) ** 0.5
```

Рис. 4.3. Метод класса

При этом `self` нужен только при определении метода. При вызове же он не указывается (рис. 4.4). На самом деле туда будет подставлен объект, который вызвал этот метод «через точку».

```

1  p1 = Point2d()
2  p1.x = 4
3  p1.y = 3
4  p2 = Point2d()
5
6  # Вызов метода, в который передается только 'other'
7  print(p1.distance(p2))

```

Рис. 4.4. Вызов метода

4.3. Специальные (магические) методы

Магические методы — это методы из некоторого определенного в языке перечня методов с особым поведением. Их названия начинаются и заканчиваются с удвоенного нижнего подчеркивания. Однако все они вызываются не по их собственному имени, а «по-особому». Далее будет показано как.

Можно создавать разные экземпляры класса с заранее заданными настройками (например, значениями для полей) с помощью **инициализатора** — метода `__init__` (рис. 4.5).

```

1  class Point2d:
2      # Здесь поля не перечисляются
3
4      def __init__(self, x, y):
5          self.x, self.y = x, y
6
7  p1 = Point2d(3, 4)

```

Рис. 4.5. Класс с инициализатором и создание инстанса

Деструктор имеет вид `__del__` (рис. 4.6). На практике деструктор используется редко, в основном для тех ресурсов, которые требуют явного освобождения памяти при удалении объекта. Не следует совершать в нем сложные вычисления.

```

1  class Point2d:
2      # ... другие поля и методы класса ...
3
4      def __del__(self):
5          pass # Деструктор пустой
6
7
8  p1 = Point2d(3, 4)
9  del p1 # Оператор del вызовет деструктор

```

Рис. 4.6. Не имеющий смысла деструктор и его вызов в коде

Если этот метод не определить оператор `del` все равно вызывается без ошибок. Здесь программистом просто указывается *дополнительное поведение*, выполняемое при удалении. Например, для дерева (или другой сложной иерархической структуры) полезно вызвать удаление всех ветвей рекурсивно.

Пользовательские объекты могут участвовать в **арифметических операциях**, если у них определены специальные методы:

- `__add__` для операции сложения (+);
- `__sub__` для операции вычитания (-);
- `__mul__` для операции умножения (*);
- `__truediv__` для операции деления (/);
- `__floordiv__` для операции целочисленного деления (//);
- `__mod__` для нахождения остатка от деления (%);
- `__pow__` для возведения в степень (**) и другие.

В скобочках указано как они будут вызываться в коде. Пример представлен на рисунке 4.7.

```
1 class Point2d:
2     # ... другие поля и методы класса ...
3
4     def __add__(self, other): # покомпонентно
5         return Point2d(self.x+other.x,
6                         self.y+other.y)
7
8
9 p1 = Point2d(4, 3)
10 p2 = Point2d(1, 1)
11
12 p3 = p1+p2 # вызов магического метода
```

Рис. 4.7. Магический метод для оператора сложения

Кроме «обычных» арифметических методов есть еще:

- «отраженные» — начинаются с `r`, например, `__radd__`, и используются когда инстанс является вторым (правым) операндом, а не первым (не слева от оператора);

- «дополненные» — начинаются с `i`, например, `__iadd__`, и используются при вызове операторов дополненного присваивания (рис. 4.8).

```
1 class Point2d:
2     # ... другие поля и методы класса ...
3
4     def __iadd__(self, other): # покомпонентно
5         return Point2d(self.x+other.x,
6                         self.y+other.y)
7
8
9 p1 = Point2d(4, 3)
10 p1 += Point2d(1, 1) # вызов магического метода
```

Рис. 4.8. Магический метод для дополненного сложения

Все эти методы должны возвращать новый полученный объект, а не менять текущий экземпляр.

Для приложения важным может оказаться реализация метода `__str__`, который используется методом `print` (рис. 4.9), позволяющему выводить информацию на консоль или в файл, а также просто для приведения к строковому типу функцией `str`.

```
1 class Point2d:
2     # ... другие поля и методы класса ...
3
4     def __str__(self):
5         return '(' + str(self.x) + ', ' + str(self.y) + ')'
6
7
8 p1 = Point2d(4, 3)
9 print(p1) # (4, 3)
```

Рис. 4.9. Метод приведения к строке

Для определения в каком случае пользовательский объект в условиях будет считаться истинным или ложным используется метод `__bool__`. При этом объект может напрямую использоваться в проверках ветвлений, циклов, тернарного оператора и операторов `and` и `or` (рис. 4.10).

```

1  class Point2d:
2      # ... другие поля и методы класса ...
3
4      def __bool__(self):
5          # (0, 0) -> False
6          # все остальное -> True
7          return bool(self.x or self.y)
8
9
10 p1 = Point2d(4, 3)
11 p2 = Point2d(1, 0)
12 p3 = Point2d(0, 0)
13
14 if (p1): print(1) # +
15 if (p2): print(2) # +
16 if (p3): print(3) # -

```

Рис. 4.10. Использование экземпляра пользовательского класса в if

Для того, чтобы определить правила для **сравнения** двух объектов пользовательского класса используется методы:

- `__eq__` для равенства (`==`);
- `__ne__` для неравенства (`!=`);
- `__lt__` для оператора меньше (`<`);
- `__le__` для оператора меньше или равно (`<=`);
- `__gt__` для оператора больше (`>`);
- `__ge__` для оператора больше или равно (`>=`).

Можно определить следующие **битовые операторы**:

- `__lshift__` битовый сдвиг влево (`<<`);
- `__rshift__` битовый сдвиг вправо (`>>`);
- `__and__` битовое И (`&`);
- `__xor__` битовое ИСКЛЮЧАЮЩЕЕ ИЛИ (`^`);
- `__or__` битовое ИЛИ (`|`).

А также дополненные и правосторонние битовые операторы.

Определив метод `__call__` можно вызывать экземпляры класса как функции (рис. 4.11).

```

1 class Counter:
2     def __init__(self):
3         self.val = 0
4     def __call__(self, value):
5         self.val += value
6
7
8 c = Counter()
9 c(5) # вызов объекта как функции
10 c(10) # вызов объекта как функции
11
12 print(c.val) # 15

```

Рис. 4.11. Функтор

С помощью него, например, можно создать «функции, запоминающие состояние между вызовами».

Для создания логики поведения пользовательского объекта как **контейнера** необходимо определить методы:

- `__len__` (длина);
- `__getitem__` (получение значения по ключу);
- `__setitem__` (установка значения для ключа);
- `__delitem__` (удаление значения);
- `__contains__` (проверка наличия).

Определение этих методов позволит использовать экземпляр данного класса как последовательность. Например, обратиться к нему по индексу с помощью квадратных скобок. Пример представлен в Приложении 1. В нем много плохого кода, но, несмотря на это, он полезен для понимания магических методов, определяющих поведение контейнера.

Для написания пользовательских **итераторов**, необходимо определить методы `__iter__` и `__next__`. Также можно определить `__reversed__` для итераторов из элементов, следующих в обратном порядке.

4.4. Инкапсуляция, наследование и полиморфизм в Python

Инкапсуляция. Все объекты в Python инкапсулируют внутри себя данные и методы работы с ними, предоставляя публичные интерфейсы для взаимодействия.

Можно выделить два подхода к значению этого термина. Инкапсуляция может быть рассмотрена как:

1. Связь данных с методами, которые этими данными управляют.
2. Набор инструментов для управления доступом к данным или методам, которые управляют этими данными.

Со вторым пунктом у языка Python проявляются некоторые особенности по сравнению с привычными объектно-ориентированными языками.

Атрибут может быть объявлен приватным (внутренним) с помощью нижнего подчеркивания перед именем (рис. 4.12), но настоящего скрывания на самом деле не происходит – все *на уровне соглашений*.

```
1 class Point2d:
2     def __init__(self, x, y):
3         self._x, self._y = x, y
4     def __str__(self):
5         return '(' + str(self._x) + ', ' + str(self._y) + ')'
6
7
8 p1 = Point2d(3, 4)
9 print(p1._x) # 3 – все равно получили доступ
```

Рис. 4.12. «Слегка закрытое» поле

Чуть лучше ситуация при использовании двойного нижнего подчеркивания перед именем поля или метода (рис. 4.13).

```
1 class Point2d:
2     def __init__(self, x, y):
3         self.__x, self.__y = x, y
4     def __str__(self):
5         return '(' + str(self.__x) + ', ' + str(self.__y) + ')'
6
7
8 p1 = Point2d(3, 4)
9 print(p1.__x) # Ошибка AttributeError – доступа нет
```

Рис. 4.13. «Почти закрытое» поле

Однако, даже так имеется способ обратиться к этому атрибуту через синтаксис «переменная. имякласса__приватныйатрибут» (рис. 4.14).

```
1 print(p1._Point2d__x) # 3 – нет ошибки
```

Рис. 4.14. Доступ к закрытому полю

Таким образом Python не обеспечивает ограниченный доступ к каким-либо переменным или методам класса. Данные, которые должны быть скрыты, на самом деле могут быть прочитаны и изменены. В Python инкапсуляция является скорее условностью, и программист должен самостоятельно заботиться о ее сохранении.

Под **наследованием** понимается возможность создания нового класса на базе существующего. При этом класс потомок будет содержать те же атрибуты и методы, что и базовый класс, но при этом его можно расширять через добавление новых методов и атрибутов.

Класс-предок указывается в круглых скобках после имени объявляемого класса (рис. 4.15).

```
1 class Point2d(object):  
2     # ... поля и методы ...  
3  
4 class Point3d(Point2d):  
5     z = 0 # Новое поле
```

Рис. 4.15. Наследование

Все классы в Python, кроме классов ошибок, явно (если указать его в круглых скобках) или неявно (без скобок как в примерах на рис. 4.1-4.14) происходит от object.

Язык программирования Python реализует как одиночное наследование, так и множественное. Родительские классы при этом указываются через запятую (рис. 4.16).

```
1 class Point3d(Point2d, Objects3d):  
2     z = 0 # Новое поле
```

Рис. 4.16. Множественное наследование

Концепция **полиморфизма** — важная часть ООП на Python. *Все методы в языке изначально виртуальные.* Это значит, что дочерние классы могут их переопределять и решать одну и ту же задачу разными путями (и для этого не используется каких-либо дополнительных ключевых слов), а конкретная реализация будет выбрана только во время исполнения программы (рис. 4.17).


```

1 class C1:
2     def Show(self):
3         print('C1', end = ' ')
4
5 class C2(C1):
6     def Show(self):
7         print('C2', end = ' ')
8
9
10 # Кортеж с объектами типа C1 и C2
11 c = (C1(), C2(), C1())
12
13 for i in c:
14     i.Show() # Вывод: C1 C2 C1

```

Рис. 4.17. Переопределение метода дочерним классом

Можно получить и доступ к методам класса-предка с помощью функции `super` (рис. 4.18).

```

1 class C2(C1):
2     def Show(self):
3         print('parents: ', end='')
4         super().Show()
5         print(' | me: ', end='')
6         print('C2', end=' ')

```

Рис. 4.18. Использование `super`

Функция `super`, возвращает объект-посредник, который делегирует вызовы метода родительскому или родственному классу, указанного `type` типа. Это полезно для доступа к унаследованным методам, которые были переопределены в классе. Необязательным параметром метода является тип, от которого начинается поиск объекта-посредника, что позволяет решить проблему выбора родительского метода при множественном наследовании.

4.5. Свойства (геттеры и сеттеры)

Декоратор `@property` превращает метод в свойство (геттер). К нему можно будет обращаться через точку без скобочек (рис. 4.19).

То есть данный метод используется как поле класса, но только на чтение (только в правой части присваивания).

```

1 class Point2d:
2     def __init__(self, x, y):
3         self.__x, self.__y = x, y
4
5     @property
6     def X(self):
7         return self.__x
8
9
10 p1 = Point2d(3, 4)
11 coord = p1.X

```

Рис. 4.19. Использование встроенного декоратора property

Декоратор **@имя.setter** позволяет написать сеттеры (рис. 4.20).

```

1 class Point2d:
2     def __init__(self, x, y):
3         self.__x, self.__y = x, y
4
5     @X.setter
6     def X(self, val):
7         self.__x = val
8
9
10 p1 = Point2d(3, 4)
11 p1.X = 10

```

Рис. 4.20. Использование встроенного декоратора property

Теперь метод может использоваться как поле на запись (в левой части присваивания).

Не стоит думать, что геттеры и сеттеры нужны только для того, чтобы в одну строчку кода возвращать или присваивать закрытое поле. Существуют, например, «вычисляемые поля» (Приложение 2). Сеттеры в хорошем коде всегда осуществляют валидацию, а также могут содержать просто более сложный код, чем в ранее представленных примерах, поскольку есть задачи, когда изменение одного поля может потребовать изменения других или выполнения каких-либо дополнительных действий.

4.6. Абстрактные классы

Абстрактный класс — очень важная концепция объектно-ориентированного программирования. Такой класс имеет следующие особенности:

1. Абстрактный класс не содержит всех реализаций методов, необходимых для полной работы, это означает, что он содержит один или несколько абстрактных методов. Абстрактный метод — это только объявление метода, без его реализации.

2. Абстрактный класс предоставляет интерфейс для подклассов, чтобы избежать дублирования кода. Не имеет смысла и невозможно создать экземпляр абстрактного класса.

3. Производный подкласс должен реализовать все абстрактные методы (если сам не является абстрактным).

В стандартную библиотеку Python включается модуль `abc` (Abstract Base Classes), добавляющий в язык абстрактные базовые классы. Свой абстрактный класс можно определить с помощью наследования от класса `ABC` (рис. 4.21).

```
1 from abc import ABC, abstractmethod
2
3 class MyBase(ABC):
4     pass
```

Рис. 4.21. Создание абстрактного класса

Абстрактные методы помечаются декоратором `@abstractmethod` (рис. 4.22).

```
1 from abc import ABC, abstractmethod
2
3 class MyBase(ABC):
4     @abstractmethod
5     def Show(self): pass
```

Рис. 4.22. Абстрактный метод абстрактного класса

При этом абстрактный класс может содержать и обычные, «реализованные» методы, которые пригодятся в классах-наследниках.

4.7. Статические поля и методы

Для создания **статических методов** в Python предназначен декоратор `@staticmethod`. У них нет параметра `self`. Статический метод не может изменять ни состояние объекта, ни состояние класса.

Статические методы могут работать только с:

- параметрами, переданными в них;
- другими статическими методами и полями;
- переменными, объявленными в самом методе.

Они не могут обращаться к обычным полям и методам через `self` (рис. 4.23).

```
1 class Point2d:
2     @staticmethod
3     def Info():
4         return 'This is 2d Point'
5
6 # Экземпляр не нужен
7 print(Point2d.Info())
```

Рис. 4.23. Статический метод

В Python можно вызвать статический метод не только по имени класса, но и по имени экземпляра объекта (однако этот объект не будет передан в метод как `self`).

Методы класса принимают класс в качестве параметра, который принято обозначать как `cls`. Он указывает на класс, а не на объект этого класса. При объявлении методов этого вида используется декоратор `@classmethod`.

Методы класса привязаны к самому классу, а не его экземпляру. Они могут менять состояние класса, что отразится на всех объектах этого класса, но не могут менять конкретный объект.

Такие методы позволяют создать альтернативный конструктор или несколько конструкторов класса.

Статических свойств, которые можно было бы сделать по аналогии со статическими методами в Python нет, но есть возможность обращаться ко всем полям по имени класса и тогда они будут использоваться как статические (рис. 4.24).

```

1 class Ant:
2     TOTAL = 0
3
4     def __init__(self, ant_type):
5         self.ant_type = ant_type
6         Ant.TOTAL += 1 # Статическое
7
8 w = Ant('Worker')
9 q = Ant('Queen')
10 print(Ant.TOTAL) # 2

```

Рис. 4.24. Обращение к полю по имени класса

Такое поле обязательно должно быть объявлено в самом классе (строка 2), а не в инициализаторе.

5. Аннотации

Python — язык с динамической типизацией, что позволяет свободно оперировать переменными разных типов. Однако, при написании кода, типы переменных, которые будут использоваться, всегда предполагаются.

Сохраняя идею динамической типизации, в современных версиях Python поддерживаются аннотации:

- типов переменных,
- полей класса,
- аргументов и возвращаемых значений функций.

Аннотации типов позволяют добавлять подсказки о типах переменных. Они используются, чтобы информировать читателя кода, каким должен быть тип переменной. Это придаёт немного статический вид коду на динамически типизированном Python. Аннотации типов просто считываются интерпретатором Python и никак более не обрабатываются.

Аннотации типов поддерживаются многими IDE для Python, которые выделяют некорректный код или выдают подсказки в процессе набора текста.

Синтаксис аннотаций: «переменная: тип = значение» (рис. 5.1).

```

1 x: int = 25
2 y: int = 'some string' # Ошибки не возникает

```

Рис. 5.1. Аннотация типа переменной

В отличие от других двух видов аннотаций, указание типа переменой полезно только в редких случаях.

Аннотации полей классов выглядят аналогично.

Используя аннотации, можно указать какие параметры ожидает пользовательская функция и какого типа будет ее результат (рис. 5.2).

```
1 def mult(a: float, b: float) -> float:  
2     return a*b;
```

Рис. 5.2. Аннотации и функция

6. Исключения

Исключения — это сценарии ошибок, которые изменяют нормальный поток выполнения программы. Процесс обработки возможных исключений называется обработкой исключений.

Если исключения не обрабатываются должным образом, программа может завершиться преждевременно. Это может привести к повреждению данных или нежелательным результатам.

Обработка исключений Python выполняется с помощью следующего набора ключевых слов:

1. Блок **try** содержит код, который может вызывать исключения или ошибки.

2. Блок **except** используется для перехвата исключений и их обработки. Код блока **except** выполняется только при возникновении соответствующего исключения. Может быть несколько блоков **except**, а также один **except** может обрабатывать несколько исключений.

3. Код блока **finally** всегда выполняется независимо от того, правильно ли выполнялась программа или возникла исключительная ситуация.

4. Код внутри блока **else** выполняется, если не возникают исключения.

Пример обработки исключения представлен на рисунке 6.1.

```

1  a = int(input('Введите делимое '))
2  b = int(input('Введите делитель '))
3  try:
4      x = a / b
5  except ZeroDivisionError as e:
6      print('Ошибка: ', e)
7  else:
8      print('Результат: ', x)

```

Рис. 6.1. Работа с исключением

Ключевое слово **as** — сохраняет ошибку в переменную.

Некоторые из встроенных классов исключений в Python:

- `ArithmeticError` — базовый класс для арифметических ошибок;
- `AttributeError` — атрибут не найден;
- `EOFError` — чтение после конца файла;
- `ImportError` — импортированный модуль не найден;
- `MemoryError` — при нехватке памяти;
- `NameError` — когда имя не найдено глобально;
- `SyntaxError` — недопустимый синтаксис;
- `TypeError` — недопустимый тип аргумента;
- `ValueError` — недопустимое значение аргумента.

Исключения можно генерировать самостоятельно — для этого нужно запустить оператор **raise** (рис. 6.2). Например, если в функцию передано значение такого типа, с которым дальнейшая работа бессмысленна и невозможна, или если просто значения недопустимы.

```

1  # Площадь треугольника
2  def tri_area(a, b, c):
3      # Сторона должна быть меньше суммы остальных
4      if a >= (b+c) or b >= (a+c) or c >= (a+b):
5          raise ValueError
6      pp = (a + b + c) / 2
7      return (pp*(pp-a)*(pp-b)*(pp-c)) ** 0.5

```

Рис. 6.2. Выбрасывание исключения

Можно создать свой тип исключений, наследуя класс от класса `Exception` (рис. 6.3.)

```

1 class MyError(Exception):
2     def __init__(self, text):
3         self.txt = text

```

Рис. 6.3. Пользовательский тип исключений

При создании своего класса исключения рекомендуется добавлять к его имени суффикс «Error».

7. Комплексные числа

Комплексное число — это любое число в форме $a + bj$, где a и b — действительные числа, а $j*j = -1$.

В Python есть несколько способов создать такое комплексное число: можно напрямую использовать синтаксис $a + bj$ для создания комплексного числа, либо использовать класс `complex` (рис. 7.1).

```

1 x = 5 + 2j
2 y = complex(5, 2)

```

Рис. 7.1. Объявление комплексного числа

Чтобы получить значения действительной (`real`) и мнимой (`imag`) частей комплексного числа, используется синтаксис вызова через точку (рис. 7.2).

```

1 print(x.real) # 5.0
2 print(x.imag) # 2.0

```

Рис. 7.2. Печать вещественной и мнимой частей по отдельности

Сопряжение находится методом `conjugate`.

Комплексные числа поддерживают арифметические операции: их можно складывать, вычитать, умножать и делить как друг на друга, так и на вещественные числа (рис. 7.3).

```

1 print(x * y) # (21+20j)
2 print(x * 2) # (10+4j)

```

Рис. 7.3. Операции с комплексными числами

Невозможно сравнивать два комплексных числа целиком: можно сравнивать только действительную и мнимую части по отдельности.

8. Длинная арифметика

При работе с целыми числами в Python, не может произойти переполнение типов, так как в языке используется длинная арифметика. Представления чисел ограничены только доступной памятью системы (рис. 8).

```
1 print(2**1000)
2
3 # Результат:
4 # 1071508607186267320948425049060001810561404811705533607443
5 # 7503883703510511249361224931983788156958581275946729175531
6 # 4682518714528569231404359845775746985748039345677748242309
7 # 8542107460506237114187795418215304647498358194126739876755
8 # 9165543946077062914571196477686542167660429831652624386837
9 # 205668069376
```

Рис. 8. Возведение в степень

Реализовано это следующим образом: число хранится в виде массива, каждый элемент этого массива — числа в системе счисления с основанием 2^{30} (1.073.741.824).

Выполнение операций сложения, вычитания и умножения, при представлении числа в виде массива, основано на операциях «в столбик». Деление и другие операции над целыми числами имеют более сложную логику.

В случаях, использования не встроенных целых чисел, а библиотек `numpy`, `scipy`, `pandas` и других, может произойти переполнение при работе с целыми числами, так как структуры, лежащие в основе этих библиотек, для более эффективного использования памяти, полагаются на соответствующие C-типы ограниченной точности.

9. Множественный выбор `match-case`

Начиная с версии языка Python 3.10 появился оператор сопоставления `match-case`. Он не только заменяет оператор `switch-case` C-подобных языков, но может выполнять и более сложные проверки (сопоставления шаблонов).

Общий синтаксис представлен на рисунке 9.1.

```

1  match x:
2      case шаблон_1:
3          блок_кода_1
4      case шаблон_1:
5          блок_кода_2
6      case _: # default
7          блок_кода_3

```

Рис. 9.1. Устройство match-case

После выполнения блока кода первого истинного выражения, последующие условия case не вычисляются, а соответствующие инструкции не выполняются.

Блок case _ не обязателен и выполнится, если ни одно условие в других case не будет верным. Он аналогичен блоку default оператора switch-case С-подобных языков программирования.

Простейший пример представлен на рисунке 9.2.

```

1  match name:
2      case 'red':
3          color = '#f00'
4      case 'green':
5          color = '#0f0'
6      case 'blue':
7          color = '#00f'
8      case _:
9          color = '#000'

```

Рис. 9.2. Простой пример match-case

Проверка состояния — это не просто точное совпадение переменной с указанным в case значением, а сопоставление с шаблоном (pattern matching statement). Это целый механизм для проверки данных, их распаковки и управления потоком выполнения.

Код для проверки **совпадения с одним из шаблонов** представлен на рисунке 9.3. Здесь «|» означает «или».

```

1  match direction:
2      case 'North' | 'East' | 'South' | 'West':
3          print('Хорошо, я пошел!')
4      case _:
5          print('Неизвестное направление...')

```

Рис. 9.3. Пример с «или»

Распаковка при проверке (см. «распаковывающее присваивание») представлена на рисунке 9.4.

```

1  x = (3, 'A', 'B', 'C')
2
3  match x:
4      case 3, *rest:
5          create_triangle(rest) # user function
6      case 4, *rest:
7          create_tetragon(rest) # user function
8      case _:
9          print('Can\'t create')

```

Рис. 9.4. Пример с распаковкой

Здесь в 4 и 6 строках происходит сравнение первого значения кортежа с числом, а весь остаток распаковывается в список `rest`. В дальнейшем, внутри данной ветки `case`, с этой переменной `rest` можно работать.

Аналогичным образом можно использовать новые переменные в строке `case`, чтобы помещать туда не сравниваемую, а манипулируемую информацию (рис. 9.5).

```

1  command = ('Inc', 25)
2  value = 5
3
4  match command:
5      case 'Inc', n:
6          value+=n
7      case 'Dec', n:
8          value-=n
9
10 print(result) #30

```

Рис. 9.5. Сравнение первого элемента и использование второго

Также данный оператор позволяет проверить **соответствие шаблону** (рис. 9.6).

```

1  match c:
2      # если c представляет из себя тройку значений
3      case (r, g, b):
4          color = '#%02x%02x%02x'%(r,g,b)
5
6      # если c — атомарное значение
7      case str_code:
8          color = str_code.lower()

```

Рис. 9.6. Проверка является ли переменная кортежем из трех

С помощью этого оператора также можно проверить **принадлежность к классу** (Приложение 3).

Все перечисленные шаблоны (и те, о которых в данном пособии не упомянуто) можно комбинировать между собой.

10. Перечисления Enum

Перечисления — это наборы символических имен, связанных с уникальными константными значениями. Они используются для создания простых пользовательских типов данных, таких как времена года, недели, виды оружия в игре, планеты, оценки или дни. По соглашению имена перечислений начинаются с заглавной буквы и употребляются в единственном числе.

Для создания перечислений в Python используется модуль `enum`. Пример перечисления представлен на рисунке 10.1. Для создания перечисления используется слово `class` и наследование от `Enum`.

```
1  from enum import Enum
2
3  class Status(Enum):
4      READY = 1
5      WAIT_END_PROCESS = 2
6      FAILURE = 0
```

Рис. 10.1. Пример перечисления

Перечисления нужны для понятности кода. Рассмотрим функцию, работающую с перечислением (рис. 10.2).

```
1  def PrintData(status):
2      if status == Status.READY:
3          print(data) # глобальная переменная
4      elif status == Status.FAILURE:
5          print('Произошел сбой')
6      else:
7          ... # do nth
8
9  PrintData(Status.READY)
```

Рис. 10.2. Использование перечисления

На рисунке 10.3. представлен код этой же функции без перечислений. Понимать данный код гораздо сложнее. Как программисту, впервые увидевшему данную программу узнать коды статусов и что подразумевает ветка `else`? Возможно они есть в документации или где-то в комментариях, возможно — только в голове у написавшего код.

```

1 def PrintData(status):
2     if status == 1:
3         print(data) # глобальная переменная
4     elif status == 0:
5         print('Произошел сбой')
6     else:
7         ... # do nth
8
9 PrintData(1)

```

Рис. 10.3. Без перечисления

Значение перечисления можно напечатать функцией `print`. Список всех значений перечисления можно получить функцией приведения к списку `list`. Проверить, содержит ли переменная значение из заданного перечисления, можно функцией `isinstance`. Получить доступ к полю из перечисления можно как по его названию, так и по его значению (рис. 10.4).

```

1 x = Status.WAIT_END_PROCESS
2 print(x) # Status.WAIT_END_PROCESS
3
4 print(list(Status))
5 # [<Status.READY: 1>, <Status.WAIT_END_PROCESS: 2>,
6 #    <Status.FAILURE: 0>]
7
8 print(isinstance(Status.READY, Status)) # True
9 print(isinstance(1, Status))           # False
10
11 print(Status['FAILURE']) # Status.FAILURE
12 print(Status(1))        # Status.READY

```

Рис. 10.4. Операции с перечислением

Вместо ручного указания числовых значений для перечисления можно использовать функцию `auto`, ее также надо импортировать из `enum` (рис. 10.5).

```

1 from enum import Enum, auto
2
3 class Positon(Enum):
4     FIRST = auto()
5     OTHER = auto()
6     LAST = auto()
7     UNKNOWN = auto()
8
9 print(list(Positon))
10 # [<Positon.FIRST: 1>, <Positon.OTHER: 2>,
11 #    <Positon.LAST: 3>, <Positon.UNKNOWN: 4>]

```

Рис. 10.5. Автоматическая генерация значений для перечисления

Стоит отметить, что значения в общем случае не обязаны быть уникальными, однако члены перечисления имеющие одинаковые значения не различимы между собой.

Для создания перечисления необязательно использовать класс. Альтернативный способ создания представляет из себя вызов функции Enum (рис. 10.6).

```
1  from enum import Enum
2
3  P = Enum('Chance',          # название
4          'LOW MEDIUM HIGH', # набор данных
5          start=1)            # для генерации их кодов
6
7  print(list(P))
8  # [<Chance.LOW: 1>, <Chance.MEDIUM: 2>, <Chance.HIGH: 3>]
```

Рис. 10.6. Функция для создания перечисления

11. Задачи на ООП и декораторы

11.1. Пример

Задача. Написать класс «автомат газированной воды», который имеет набор газировок и цен, соответствующих им. Он может предоставить всю информацию о продаваемых газировках в виде списка строк, добавить новый вид газировки (перегрузив оператор +=), а также проверить, хватит ли введенной суммы, для покупки выбранной газировки. Должна быть предусмотрена возможность ввода и вывода из файла и с консоли.

Поля сделать закрытыми и с геттерами; сеттеры дописать по мере необходимости. У класса должны быть определены инициализатор и метод приведения к строке.

Написать декоратор, который к каждому элементу-строке списка приклеивает символ «\$» в конце. Применить его к методу, возвращающему список газировок.

Создать консольное приложение с меню, демонстрирующее работу кода.

Решение:

Декоратор

```
def add_bucks(func):
    def wrapper(*args, **kwargs):
        return [i+'$' for i in func(*args, **kwargs)]
    return wrapper
```

Класс

```
class SodaMachine:
    # Инициализатор
    def __init__(self):
        self.__soda = dict() # названия и цены

    # Добавление
    def __iadd__(self, val):
        self.__soda[val[0]] = val[1]
        return self

    # Возвращает список газировки
    @add_bucks
    def get_all_soda(self):
        return [str(k)+' ': str(v) for k, v in
                self.__soda.items()]

    # Достаточно ли суммы
    def enough(self, soda, money):
        # Нет такой газировки
        if not (soda in self.__soda.keys()):
            return False
        # Газировка есть
        return money >= self.__soda[soda]

    # Приведение к строке
    def __str__(self):
        res = ''
        for k, v in self.__soda.items():
            res += str(k) + ': ' + str(v) + '\n'
        return res[:-1]

    # Геттер
    @property
    def soda(self):
        # Возвращается копия чтобы реальный словарь нельзя
        # было испортить извне
        return self.__soda.copy()
```

Консольное приложение

```
# Добавить
def add(sm):
    name = input('Введите название: ')
    price = int(input('Введите цену: '))
    sm += (name, price)
    print('Добавлено\n')

# Вывести в файл или на консоль
# (за выбор отвечает параметр file)
def show(sm, file=None):
    print('Как вывести?')
    print('1) Используя __str__()')
    print('2) Используя get_all_soda()')
    print('3) Используя геттер soda')
    choise = int(input())
    if 1 == choise:
        print(sm, end='', file=file)
    if 2 == choise:
        print(sm.get_all_soda(), end='', file=file)
    if 3 == choise:
        print(sm.soda, end='', file=file)
    if file: # если был вывод в файл
        print('Готово\n')

# Проверка хватает ли средств
def check_price(sm):
    name = input('Введите название: ')
    price = int(input('Введите сумму: '))
    if sm.enough(name, price):
        print('Средств достаточно\n')
    else:
        print('Средств не хватает или нет такого напитка\n')

# Чтение из файла (только если сохранено через __str__())
def from_file(filename):
    result = SodaMachine()
    count = 0
    with open(filename, 'rt', encoding='utf-8') as f:
        for string in f:
            data = string.split(': ')
            result += (data[0], int(data[1]))
            count += 1
    print('Прочитано %d записей\n' % count)
    return result

# Вывод пунктов меню
def show_menu():
    print('Выберите действие')
```



```

print('[1] Напечатать')
print('[2] Прочитать из файла')
print('[3] Добавить значение с клавиатуры')
print('[4] Проверить хватает ли денег')
print('[5] Сохранить в файл')
print('[0] Завершить задачу')

# Основное меню
def main():
    s = SodaMachine()
    while(True):
        show_menu()

        choise = int(input())
        if 1 == choise:
            show(s)
            print('\n')
        elif 2 == choise:
            filename = input('Введите имя файла: ')
            s = from_file(filename)
        elif 3 == choise:
            add(s)
        elif 4 == choise:
            check_price(s)
        elif 5 == choise:
            filename = input('Введите имя файла: ')
            w_file = open(filename, 'wt', encoding='utf-8')
            show(s, file=w_file)
        elif 0 == choise:
            break
        else:
            print('> Выберите пункт из списка!')

main()

```

11.2 Условия задач

Задача должна быть выполнена на языке Python, используя пройденный материал. Условия задачи и автор должны быть указаны в комментариях. Также должны быть «обычные» комментарии в коде.

Создать консольное приложение с меню, которое будет демонстрировать все действия, которые можно выполнить над указанным классом. Задача не должна завершаться после одного действия — вместо этого каждый раз должна возвращаться в меню до выбора пункта «завершить».

Пользователь (преподаватель) должен понимать что ему нужно делать из консоли, а не из условия задачи, кода программы или ваших устных подсказок.

Во всех задачах должна быть предусмотрена возможность ввода и вывода из файла и с консоли. Задачи должны быть разбиты на функции.

Классы вынести в отдельный модуль. Поля сделать закрытыми и с геттерами; сеттеры дописать по мере необходимости. У класса должны быть определены инициализатор и метод приведения к строке.

Декорирование происходит через @.

Условия:

1. Написать декоратор, сохраняющий название функции (с помощью `__name__`) в список. Список поступает как параметр декоратора.

Написать класс «Item» с полями «название», «описание», «цена», «число зарядов». Перегрузить операторы «+=» и «-=», изменяющие число зарядов на заданное целочисленное значение.

Программа должна содержать список айтемов, которые можно добавлять, удалять, изменять число зарядов и печатать. Декоратор применить ко всем методам класса.

2. Написать декоратор_1, который приводит строку-результат функции к верхнему регистру. Написать декоратор_2, который добавляет кавычки-елочки вокруг строки-результата.

Написать класс «Truck» с полями «марка», «модель», «мощность двигателя», «объем бака», «текущий уровень топлива». Перегрузить операторы «+=» и «-=», изменяющие текущий уровень топлива на заданное вещественное значение. Перегрузить операторы сравнения, сравнивающие два тягача по мощности двигателя.

Программа должна содержать список тягачей, которые можно добавлять, удалять, изменять уровень топлива и печатать. Декораторы 1 и 2 применить ко геттеру марки. Декоратор 2 применить к геттеру модели.

3. Написать декоратор с двумя параметрами: значение для True, значение для False. Он заменяет логическое значение-результат функции на значение одного из параметров.

Написать класс «Пациент» с полями «ФИО», «пол» (bool), «диагноз», «жив ли» (bool).

Программа должна содержать список пациентов, которых можно добавлять, удалять и печатать. Декоратор применить к геттерам «пол» и «жив ли».

4. Написать декоратор, который автоматически подставляет значение во второй параметр функции, тем самым уменьшая число ее аргументов. Декоратор подставляет строку-цвет: один из семи цветов радуги — при очередном вызове цвет автоматически выбирается «следующим» (т.е. к-о-ж-з-г-с-ф-к...). Для этого декоратор должен «помнить» предыдущее значение. Декоратор вынести в отдельный модуль.

Написать класс «Рыцарь» с полями «имя», «цвет знамени» и «титул». Добавить сеттер цвета знамени.

Программа должна содержать список рыцарей, которых можно добавлять, удалять и печатать. Декоратор применить к инициализатору рыцаря и сеттеру знамени.

5. Написать декоратор «удача» с параметрами N и M: с шансом N% увеличивает результат в M раз.

Написать класс «Гладиатор» с полями «имя», «здоровье» и «урон». Декорировать геттер урона.

Задача должна мочь настраивать двух гладиаторов и устраивать поединок между ними.

6. Написать декоратор с параметром-коэффициентом. Он переводит результат из одной величины в другую.

Написать класс «Округ» с полями «название» и «площадь». Переопределить операторы «+=», «-=», «*=» и «/=», изменяющие его площадь на вещественное число.

Программа должна содержать список округов, которые можно добавлять, удалять, изменять площадь и печатать. Декоратором, который переводит м² в попугай² (один попуг = 19,8см). декорировать геттер площади.

7. Написать декоратор, который помимо основного действия печатает «Вы осмелились потревожить .name от первого аргумента».

Написать класс «Бог» с полями «имя», «стихия». Добавить метод «выслушать молитву», который принимает строку и ничего с ней не делает. Переопределить операторы «==» и «!=», сравнивающие богов по имени.

Написать класс «Пантеон богов», который содержит список богов. Переопределить методы «+= Бог» для добавления и «-= Бог» для удаления, если такой бог найден.

Программа должна содержать пантеон богов, добавлять, удалять богов, печатать пантеон. Декоратор применяется ко всем методам класса «Бог» кроме геттеров и инициализатора.

8. Написать декоратор, который число заменяет на приближенное значение, превращая в строку вида «число единица_измерения (миллионы, тысячи, сотни или десятки)» (то есть, например, 273407 на «273 тысячи», 215 на «2 сотни»).

Написать класс «Стихийное бедствие» с полями «тип» (например, землетрясение или тайфун), «место», «число погибших». Декорировать геттер на «число погибших».

Написать дочерний класс «Техногенная катастрофа», который имеет дополнительное поле «виновник».

Программа должна содержать список бедствий и катастроф, которые можно добавлять, удалять и печатать.

9. Написать декоратор, который в строке-результате функции переставляет первое слово в конец.

Написать класс «Джедай» с полями «имя», «цвет меча», «навык» (enum: Впитывание силы, Лечение, Воздействие на разум, Защита, Похищение силы, Молния, Удушение, Темная ярость), «коронная фраза». Добавить сеттер на навык.

Программа должна содержать список джедаев, которые можно добавлять, удалять, менять навык и печатать. Применить декоратор к геттеру коронной фразы.

10. Написать декоратор с параметром «суффикс», который превращает результат в строку, добавляя суффикс в конец.

Написать класс «Геокоордината» с полями «градусы», «минуты», «секунды». Декорировать все геттеры.

Класс «Метеодатчик» с полями «долгота» и «широта» (геокоординаты), «ID», «температура», «влажность». Декорировать геттеры влажности (%) и температуры (°).

Программа должна содержать список метеодатчиков, которые можно добавлять, удалять и печатать.

11. Написать декоратор, который округляет число до целого.

Написать класс «Аттестат», который хранит ФИО ученика и набор пар «предмет, оценка». Он содержит метод, возвращающий средний балл по предметам (декорировать его). Для добавления оценки за предмет перегрузить оператор «+=». Реализовать магические методы класса-контейнера (len, getitem, setitem, delitem, contains).

Программа должна мочь создавать аттестат и работать с ним.

12. Написать декоратор_1, который функцию от 2 аргументов, превращает в функцию от 1 аргумента, автоматически подставляя вместо первого π из математического модуля.

Написать декоратор_2, который превращает число-результат в строку, состоящую из этого числа с точностью ровно 6 знаков после запятой.

Написать статический класс «Калькулятор площадей» с методами вычисления площадей правильных: 3, 4, 5 и 6-угольников по их стороне; площади круга, которая принимает π и радиус. Декоратор 1 применить к методу нахождения площади круга. Декоратор 2 применить ко всем методам.

Задача должна демонстрировать все возможности класса.

13. Написать декоратор, который переводит год в век (целочисленный). Написать декоратор, который переводит арабское число (от – 40 до +40) в римское с подписью «до н.э.» для отрицательных значений.

Написать класс «Историческое событие» с полями «название» (str) и «год» (int). Все поля можно изменять. Применить оба декоратора к геттеру года.

Программа должна содержать список событий, которые можно добавлять, удалять, редактировать и печатать.

Список литературы

1. Лутц М. Изучаем Python Том 1. 5-е издание. – ООО “Диалектика”, 2019. - 833 с.
2. Python. Статические свойства (поля, элементы класса) кода – URL: <https://dev-gang.ru/article/kommentirovanie-python-kodaauf6lgv2vv/> (дата обращения: 26.10.2023).
3. Приватные методы без нижнего подчеркивания и интерфейсы в Python – URL: <https://habr.com/ru/articles/443192/> (дата обращения: 26.10.2023).
4. Статические переменные и методы в Python – URL: <https://pythonpip.ru/osnovy/staticheskie-peremennye-i-metody-v-python> (дата обращения: 26.10.2023).
5. Перегрузка операторов – URL: <https://pythonworld.ru/osnovy/peregruzka-operatorov.html> (дата обращения: 26.10.2023).

6. Объектно-ориентированное программирование в Python – URL: <https://codechick.io/tutorials/python/python-what-is-oop> (дата обращения: 26.10.2023).
7. Базовые математические функции – URL: <https://proproprogs.ru/modules/numpy-bazovye-matematicheskie-funkcii> (дата обращения: 01.10.2023).
8. Изменить размер рисунка в Matplotlib – URL: <https://dev-gang.ru/article/izmenit-razmer-risunka-v-matplotlib-9btqsjj4ff/> (дата обращения: 01.10.2023).
9. Как использовать функцию NumPy `arange()` в Python – URL: <https://andreyex.ru/programirovanie/python/kak-ispolzovat-funktsiyunumpy-arange-v-python/> (дата обращения: 01.10.2023).
10. NumPy, часть 1: начало работы – URL: <https://pythonworld.ru/numpy/1.html> (дата обращения: 01.10.2023).
11. Библиотека Numpy. Полезные инструменты – URL: <https://devpractice.ru/numpy-useful-functions/> (дата обращения: 01.10.2023).
12. Руководство по использованию Python-библиотеки NUMPY – URL: <https://pythonru.com/biblioteki/rukovodstvo-po-ispolzovaniju-python-biblioteki-numpy> (дата обращения: 01.10.2023).
13. Гипербола и биквадратные уравнения – URL: https://www.mathnet.ru/php/archive.phtml?wshow=paper&jrnid=mo&paperid=627&option_lang=rus (дата обращения: 26.10.2023).
14. Уравнения кривых. Лемниската Бернулли – URL: <https://www.calc.ru/Uraveniya-Krivykh-Lemniskata-Bernulli.html> (дата обращения: 26.10.2023).
15. Кардиоида – URL: <http://school-collection.edu.ru/catalog/res/5c484b99-e427-4a7b-89a6-60e64f6c7426/view/> (дата обращения: 30.09.2023).
16. Биквадратная кривая – URL: <https://mash-xxl.info/info/28249/> (дата обращения: 26.10.2023).
17. Полярные цветы – URL: <http://blog.kislenko.net/show.php?id=1891> (дата обращения: 26.10.2023).
18. Декораторы – URL: https://ru.hexlet.io/courses/python-functions/lessons/decorators/theory_unitc (дата обращения: 26.10.2023).
19. Функция `hex()` в Python, число в шестнадцатеричную строку – URL: <https://docs-python.ru/tutorial/vstroennye-funktsii-interpretatora-python/funksiya-hex/> (дата обращения: 26.10.2023).
20. Соответствие структуре шаблона, конструкция `match/case` – URL: <https://docs-python.ru/tutorial/tsikly-upravlenie-vetvleniem-python/konstruktsiya-match-case/> (дата обращения: 26.10.2023).
21. Конструкция `match-case` в Python – URL: <https://pythonist.ru/konstruktsiya-match-case-v-python-polnoe-rukovodstvo/> (дата обращения: 26.10.2023).

22. Перечисления enum в Python – URL: <https://pythonru.com/osnovy/perechisleniya-enum-v-python> (дата обращения: 26.10.2023).
23. Matplotlib Pie Charts – URL: https://www.w3schools.com/python/matplotlib_pie_charts.asp (дата обращения: 18.01.2025).
24. Python декораторы на максималках. Универсальный рецепт по написанию и аннотированию от мала до велика – URL: <https://habr.com/ru/articles/750312/> (дата обращения: 18.01.2025).
25. Магический метод __call__. Функторы и классы-декораторы – URL: https://proporprogs.ru/python_oop/magicheskiy-metod-call-funktory-i-klassy-dekoratory (дата обращения: 18.01.2025).
26. Операция вызова: __call__ в Python – URL: https://pythonlearn.ru/perezagruzka-operatorov-python/operaciya-vyzova-__call__-v-python/ (дата обращения: 18.01.2025).
27. Немного примеров match/case в Python 3.10 – URL: <https://habr.com/ru/articles/585216/> (дата обращения: 18.01.2025).

Приложение 1. Код пользовательского контейнера

```
# Класс выдающий себя за массив размером 5,  
# но при этом хранящий 5 отдельных полей  
  
class Array5:  
    def __init__(self):  
        self.a = self.b = self.c = self.d = self.e = 0  
  
    # Длина контейнера  
    def __len__(self):  
        return 5  
  
    # Получить по индексу  
    def __getitem__(self, i):  
        if not(-5 <= i < 5):  
            raise ValueError  
        # Такой код писать очень плохо:  
        if i==0 or i==-5:  
            return self.a  
        elif i==1 or i==-4:  
            return self.b  
        elif i==2 or i==-3:  
            return self.c  
        elif i==3 or i==-2:  
            return self.d  
        return self.e  
  
    # Присвоить по индексу  
    def __setitem__(self, i, val):  
        if not(-5 <= i < 5):
```



```

        raise ValueError
    # Еще плохого кода:
    if i==0 or i==5:
        self.a = val
    elif i==1 or i==4:
        self.b = val
    elif i==2 or i==3:
        self.c = val
    elif i==3 or i==2:
        self.d = val
    else:
        self.e = val

# Удалить по индексу
def __delitem__(self, i):
    # Удалить не может, только обнуляет
    self.__setitem__(i, 0)
    # Явный вызов (по имени) магического
    # метода тоже возможен

# Проверка вхождения in
def __contains__(self, val):
    if val==self.a:
        return True
    if val==self.b:
        return True
    if val==self.c:
        return True
    if val==self.d:
        return True
    if val==self.e:
        return True
    return False

def __str__(self):
    return '[' + str(self.a) + ', ' +
        str(self.b) + ', ' +
        str(self.c) + ', ' +
        str(self.d) + ', ' +
        str(self.e) + ']'

# ----- Использование -----
x = Array5()
x[0] = 10
x[1] = 44
x[2] = 26
print(x)      # [10, 44, 26, 0, 0]

print(x[-5]) # 10

```

```
print(x[1])    # 44

del x[0]
print(x)       # [0, 44, 26, 0, 0]
```

Приложение 2. Демонстрация property

Код треугольника, задаваемого двумя сторонами и углом между
 # ними. Однако программист, использующий его, равноценно
 # может получать значения любой из трех сторон не разбираясь
 # во внутреннем устройстве

```
import math as m

class Triangle:
    def __init__(self, s1, s2, ang):
        self.__s1, self.__s2 = s1, s2
        self.__ang = ang

    @property
    def Side1(self):
        return self.__s1

    @property
    def Side2(self):
        return self.__s2

    @property
    def Side3(self): # Вычисляемое поле
        a = self.Side1
        b = self.Side2
        ang = self.__ang
        return m.sqrt(a*a + b*b - 2*a*b*m.cos(ang))

# ----- Использование -----
t = Triangle(3, 4, m.pi/2)
print(t.Side1) # 3
print(t.Side2) # 4
print(t.Side3) # 5.0
```

Приложение 3. Оператор match-case с классами

Оператор сопоставления проверяет не только отношение к типу,
 # но и значения полей. Сопоставляемые значения могут быть
 # как константными, например _k1=4,
 # так и не конкретизированными, обрабатываемыми
 # в соответствующей ветке значениями, как в коде ниже

```

class Triangle:
    def __init__(self, a, b, c):
        self._a = a
        self._b = b
        self._c = c

class RightTriangle:
    def __init__(self, k1, k2):
        self._k1 = k1
        self._k2 = k2

def herons_formula(a, b, c):
    pp = (a+b+c)/2
    return (pp*(pp-a)*(pp-b)*(pp-c)) ** 0.5

#x = RightTriangle(4, 3)
x = Triangle(4, 3, 5)

match x:
    case RightTriangle(_k1=side1, _k2=side2):
        print('Area is ', side1*side2/2)
    case Triangle(_a=side1, _b=side2, _c=side3):
        print('Area is ', herons_formula(side1, side2, side3))
    case _:
        print('Can\'t handl')

# Результат: "Area is 6.0"

```